

Informe Final del Sistema

Documentación técnica, decisiones de diseño, pruebas y transferencia de conocimiento

Equipo de desarrollo

Martín Ramírez Espinosa

Mateo Almeida Gómez

28 de mayo de 2026

Índice general

Nota para los autores	8
1. Introducción general	9
2. Análisis de requerimientos, restricciones y uso	10
3. Diseño general del sistema	11
4. Hardware	12
4.1. Propósito del capítulo	12
4.2. Contexto dentro del sistema general	12
4.3. Desarrollo técnico	15
4.3.1. Arquitectura física del nodo	15
4.3.2. Ruta RF y mux de antenas	16
4.3.3. Árbol de potencia	17
4.4. Entradas, salidas e interfaces	19
4.5. Decisiones de diseño o implementación	19
4.6. Problemas presentados y ajustes realizados	20
4.7. Validación, pruebas o evidencias	21
4.8. Aprendizajes y transferencia de conocimiento	21
4.9. Limitaciones	22
4.10. Recomendaciones específicas	22
5. Software del sensor	23

5.1. Resumen	23
5.1.1. Proposito general	23
5.1.2. Objetivos especificos	23
5.2. Introduccion y arquitectura general	24
5.2.1. Servicios y modos de operacion	25
5.3. Bloques o partes del codigo	25
5.3.1. Adquisicion	25
5.4. Uso de buffers	26
5.5. Relacion con el backend	26
5.5.1. Preproceso	27
5.6. Balance IQ	27
5.7. Correccion de componente DC	27
5.7.1. Proceso	28
5.8. Estimacion de PSD	28
5.9. Demodulacion FM y AM	28
5.10. Filtrado	29
5.11. Trabajo multihilo	29
5.11.1. Postproceso	29
5.12. Remocion del pico DC en PSD	29
5.13. Calibracion en frecuencia	30
5.13.1. Idea principal	30
5.13.2. Uso de varias emisiones	31
5.13.3. Aplicacion de la correccion	31
5.14. Orquestador	31
5.14.1. Responsabilidades principales	31
5.14.2. Modos controlados por el orquestador	32
5.15. Campanas	32
5.16. PSD en tiempo real	32
5.17. PSD y audio en tiempo real	32

5.18. Calibracion	32
5.19. Protocolos y procedimientos nombrados	33
5.20. Flujo completo de funcionamiento	33
6. Plataforma web en la nube	35
6.1. Propósito del capítulo	35
6.2. Contexto dentro del sistema general	35
6.3. Desarrollo técnico	36
6.3.1. Frontend	36
6.3.2. Backend	38
6.3.3. Base de datos	41
6.3.4. Despliegue	42
6.4. Entradas, salidas e interfaces	42
6.4.1. Entradas	42
6.4.2. Salidas	43
6.4.3. Interfaces	43
6.5. Decisiones de diseño o implementación	44
6.5.1. Separación frontend/backend con API REST	44
6.5.2. WebSocket sobre polling para tiempo real	44
6.5.3. PostgreSQL y TimescaleDB para datos de series temporales	45
6.5.4. Microservicio de postprocesamiento en Python	45
6.5.5. Separación de canales WebSocket para datos y audio	46
6.6. Problemas presentados y ajustes realizados	46
6.7. Validación, pruebas o evidencias	49
6.8. Aprendizajes y transferencia de conocimiento	49
6.9. Limitaciones	51
6.10. Recomendaciones específicas	52
6.11. Repositorios	53
7. Análisis espectral y localización en la nube	54

7.1. Propósito del capítulo	54
7.2. Contexto dentro del sistema general	54
7.3. Arquitectura del módulo de postprocesamiento	55
7.4. Flujo general de procesamiento	57
7.5. Entradas del sistema	57
7.6. Salidas del sistema	59
7.7. Modos de operación	60
7.7.1. Detección automática de emisiones	61
7.7.2. Análisis por picos solicitados	61
7.7.3. Modo de cumplimiento	62
7.8. Análisis espectral realizado: frecuencia central, ancho de banda, potencia y MER/BER	62
7.8.1. Frecuencia central	62
7.8.2. Ancho de banda	63
7.8.3. Potencia de la emisión	64
7.8.4. MER y BER cuando aplica	65
8. Protocolos de prueba en nube y borde	67
9. Integración general del sistema	68
10. Conclusiones generales	69
11. Recomendaciones	70
12. Referencias	71
A. Técnica de desarrollo en grupo usando agentes IA	72
A.1. Propósito del capítulo	72
A.2. Contexto dentro del sistema general	72
A.3. Desarrollo técnico: Ecosistema de Agentes CLI	72
A.3.1. Instalación y Configuración	73

A.3.2. Arquitectura Multi-Agente	73
A.4. Decisiones de diseño: Metodología de Prompting	75
A.5. Problemas presentados y ajustes realizados	75
A.6. Validación y Evidencias	75
A.7. Aprendizajes y transferencia de conocimiento	76
A.8. Recomendaciones específicas	76
B. Diagramas adicionales	78
C. Fragmentos de código relevantes	79
D. Manuales o guías de uso	80
E. Resultados de pruebas	81

Índice de figuras

4.1. Vista del conjunto físico del nodo ANE2. Fuente: elaboración propia. . . .	14
4.2. Módulos principales del hardware ANE2. Fuente: elaboración propia. . . .	16
4.3. Detalle esquemático de rutas RF, filtros y etapa tipo mux/switch en Mon-RaF2. Fuente: elaboración propia.	17
A.1. Diseño conceptual del sistema multi-agente: Relación entre agentes y el flujo de decisión. Fuente: Elaboración propia.	74
A.2. Roles asignados y estados de diseño en el flujo de trabajo IA. Fuente: Elaboración propia.	76

Índice de tablas

4.1. Inventario funcional de componentes de hardware del nodo ANE2.	15
4.2. Criterios mínimos para una versión robusta del árbol de potencia.	18
4.3. Interfaces principales del hardware ANE2.	19
4.4. Decisiones de diseño de hardware y justificación.	19
4.5. Problemas presentados, decisiones y ajustes realizados en hardware.	20
4.6. Resumen de telemetría eléctrica y térmica inspeccionada.	21
6.1. Problemas presentados, decisiones y ajustes realizados en la plataforma web.	47
6.2. Repositorio asociado al frontend de la plataforma web.	53
6.3. Repositorio asociado al backend de la plataforma web.	53
7.1. Archivos principales del módulo de postprocesamiento espectral.	56
7.2. Entradas principales consideradas en este avance del módulo.	58
7.3. Salidas generales del procesamiento.	59
7.4. Campos principales dentro de cada resultado.	59
7.5. Modos de operación del módulo de postprocesamiento.	61
7.6. Campos asociados a la frecuencia central.	63
7.7. Campos asociados al ancho de banda.	64
7.8. Campos asociados a potencia.	65
7.9. Campos asociados a MER/BER cuando la estimación aplica.	66
A.1. Componentes de un Prompt de alta calidad.	75
A.2. Problemas en la colaboración con IA y ajustes aplicados.	75

Nota para los autores

Este documento corresponde al **informe final del sistema**. El archivo `main.tex` es el archivo maestro y debe ser modificado únicamente por el integrador del documento.

Cada autor debe trabajar únicamente en el archivo correspondiente a su capítulo dentro de la carpeta `section/`. La estructura, reglas de escritura y criterios de calidad se encuentran en el archivo `README.md`.

Cada capítulo técnico debe documentar:

- Propósito del capítulo.
- Contexto dentro del sistema general.
- Desarrollo técnico.
- Entradas, salidas e interfaces.
- Decisiones de diseño o implementación.
- Problemas presentados y ajustes realizados.
- Validación, pruebas o evidencias.
- Aprendizajes y transferencia de conocimiento.
- Limitaciones.
- Recomendaciones específicas.

Si se menciona software, el código fuente completo no debe incluirse en este informe. Debe explicarse la función del software y enlazarse el repositorio real correspondiente. Si el repositorio no existe todavía, debe escribirse explícitamente: *Repositorio pendiente de publicación*.

Antes de la versión final no debe quedar ninguna marca `\pendiente{}` en el documento.

Capítulo 1

Introducción general

Esta sección debe presentar el contexto del proyecto, el problema que se busca resolver y la motivación del sistema. También sirve para explicar, de forma breve, cuál es el alcance del documento y cómo se relacionan sus partes principales.

Como placeholder, este texto puede reemplazarse posteriormente por una narrativa más formal que describa el objetivo del sistema, los actores involucrados y el valor técnico de la solución propuesta.

Puntos sugeridos para completar

- Descripción del problema o necesidad principal.
- Objetivo general del sistema.
- Alcance del documento técnico.
- Resumen corto de la arquitectura o módulos principales.
- Relación entre borde, nube y análisis posterior.

Capítulo 2

Análisis de requerimientos, restricciones y uso

Esta sección debe consolidar los requerimientos funcionales y no funcionales del sistema, así como las restricciones de hardware, software, conectividad, presupuesto, tiempo o entorno de despliegue que condicionan el diseño.

El placeholder actual deja definido el espacio donde luego podrán incluirse tablas, listas o descripciones de casos de uso que permitan justificar decisiones de arquitectura y priorización técnica.

Puntos sugeridos para completar

- Requerimientos funcionales del sistema.
- Requerimientos no funcionales: latencia, disponibilidad, seguridad, escalabilidad.
- Restricciones de adquisición, procesamiento y transmisión de datos.
- Limitaciones del entorno operativo o experimental.
- Casos de uso o escenarios de operación.

Capítulo 3

Diseño general del sistema

En esta sección debe describirse la arquitectura general del sistema y la forma en que interactúan sus componentes. El objetivo es mostrar una visión integrada del flujo de datos, las interfaces entre módulos y la justificación de las decisiones de diseño.

Por ahora, este bloque funciona como placeholder para incorporar diagramas, descripciones de componentes, contratos de comunicación y criterios de integración entre borde y nube.

Puntos sugeridos para completar

- Arquitectura general y diagrama de alto nivel.
- Componentes principales y sus responsabilidades.
- Flujo de datos entre sensores, procesamiento en borde y servicios en nube.
- Interfaces, protocolos o formatos de intercambio.
- Decisiones de diseño y trade-offs técnicos.

Capítulo 4

Hardware

4.1. Propósito del capítulo

Esta sección documenta el hardware del nodo de sensado espectral ANE2 con un criterio de transferencia de conocimiento. La intención no es solamente listar piezas, sino explicar cómo se organiza el montaje físico, qué función cumple cada módulo, cuáles interfaces deben respetarse y qué decisiones deben mantenerse en una nueva versión del producto.

La idea técnica central es la alimentación. La experiencia de integración mostró que concentrar la entrega de energía de los periféricos en la Raspberry Pi 5 reduce el margen operativo del nodo. Un diseño derivado o una versión posterior debe incluir un **árbol de potencia externo, sobredimensionado y medible**, capaz de alimentar cómputo, SDR, LTE/GNSS, mux RF y futuras extensiones sin depender del margen de los puertos de la Raspberry Pi.

4.2. Contexto dentro del sistema general

ANE2 es un nodo de borde para monitoreo espectral. En campo, el hardware debe recibir señales RF, seleccionar o enrutar la entrada de antena, digitalizar la señal mediante SDR, ejecutar procesamiento local, obtener ubicación y mantener conectividad con la plataforma. Por tanto, el hardware cumple cuatro responsabilidades:

- **Entrada y selección RF:** las antenas llegan a la tarjeta MonRaF2, donde se concentran rutas RF, filtros y una etapa tipo mux/switch controlada por GPIO.
- **Adquisición SDR:** el HackRF One opera como receptor y entrega muestras IQ al computador de borde.

- **Cómputo de borde:** la Raspberry Pi 5 ejecuta control, telemetría, adquisición, procesamiento y comunicación con servicios superiores.
- **Conectividad y ubicación:** el módulo SIM7600G-H aporta LTE y GNSS/GPS para comunicación y georreferenciación.

La Figura 4.1 presenta el conjunto físico del nodo. La lectura correcta del montaje es: antenas hacia MonRaF2, selección o acondicionamiento RF en MonRaF2, adquisición con HackRF One, procesamiento en Raspberry Pi 5 y comunicación por LTE/GNSS.

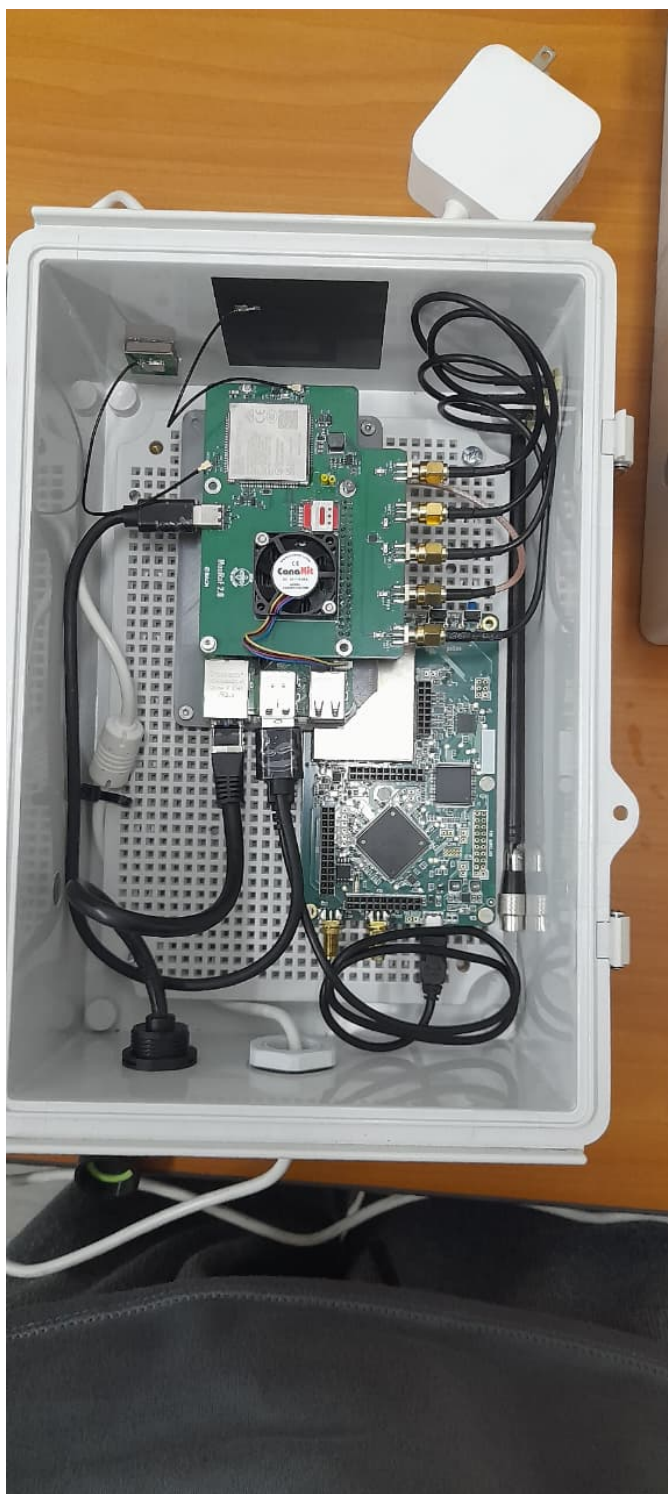


Figura 4.1: Vista del conjunto físico del nodo ANE2. Fuente: elaboración propia.

4.3. Desarrollo técnico

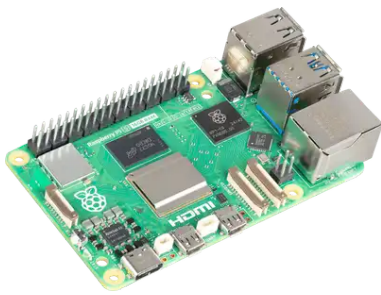
4.3.1. Arquitectura física del nodo

El nodo se compone de una Raspberry Pi 5, una tarjeta HackRF One y una PCB MonRaF2. La Raspberry Pi 5 coordina la ejecución de software, el HackRF One realiza la recepción SDR y MonRaF2 concentra conectividad, geolocalización, señales de control, protecciones y rutas RF.

Tabla 4.1: Inventario funcional de componentes de hardware del nodo ANE2.

Componente	Función principal	Criterio de integración
Raspberry Pi 5	Computador de borde	Ejecuta control, telemetría, adquisición, procesamiento y comunicación. No debe tratarse como fuente principal para todos los periféricos en una versión robusta.
HackRF One	Receptor SDR	Digitaliza señales RF como muestras IQ. Debe recibir la ruta RF ya seleccionada/acondicionada y requiere una alimentación estable para evitar fallas bajo carga.
MonRaF2	Tarjeta de integración	Integra LTE/GNSS, UART/USB/GPIO, protecciones, filtros y una ruta RF con función tipo mux/switch. Es el punto de terminación de antenas y control de rutas RF.
SIM7600G-H	LTE/GNSS	Proporciona datos celulares y ubicación. Su consumo y transitorios deben considerarse dentro del árbol de potencia.
Antenas RF, LTE y GNSS	Interfaz electromagnética	Se conectan al subsistema de MonRaF2. La conexión no debe documentarse como antena directa al HackRF, porque el diseño incluye una etapa de selección/acondicionamiento RF.
Árbol de potencia externo	Distribución de energía	Debe proveer rieles y margen para Raspberry Pi 5, HackRF, MonRaF2, LTE/GNSS y extensiones futuras. Es un requisito de diseño, no una mejora opcional.

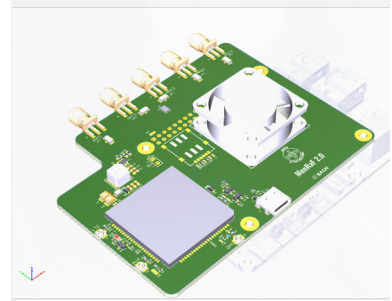
La Figura 4.2 muestra los tres módulos principales del nodo. Su separación ayuda a diagnosticar fallas: no es lo mismo un problema de ruta RF, de alimentación, de procesamiento, de enlace celular o de ubicación.



(a) Raspberry Pi 5.



(b) HackRF One.



(c) MonRaF2.

Figura 4.2: Módulos principales del hardware ANE2. Fuente: elaboración propia.

4.3.2. Ruta RF y mux de antenas

La ruta RF no debe describirse como una antena conectada directamente al HackRF. El esquemático de MonRaF2 muestra redes de antena como MAIN_ANT, GNSS_ANT y AUX_ANT, además de conectores RF, filtros LPF y una etapa de conmutación/mux RF controlada por señales GPIO. En términos operativos, MonRaF2 actúa como la tarjeta que recibe y ordena las rutas de antena antes de que la señal sea usada por el subsistema de adquisición o por el módulo celular/GNSS.

El flujo físico recomendado queda entonces:

1. Las antenas externas se conectan a MonRaF2.
2. MonRaF2 selecciona, protege o acondiciona rutas RF mediante su red de filtros, conectores y mux/switch RF.
3. La ruta RF seleccionada alimenta el subsistema correspondiente: adquisición SDR, LTE o GNSS, según el modo de operación.
4. La Raspberry Pi 5 controla la operación y recibe los datos del HackRF por USB, pero no debe asumirse como concentrador físico de antenas.

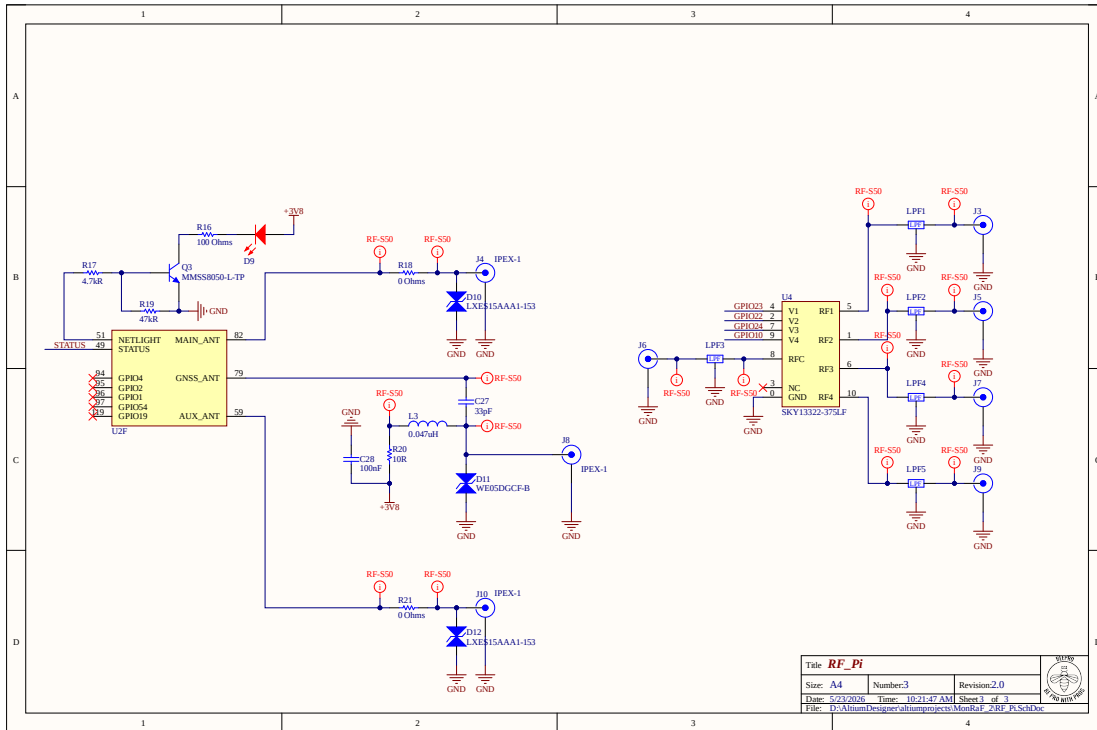


Figura 4.3: Detalle esquemático de rutas RF, filtros y etapa tipo mux/switch en MonRaF2. Fuente: elaboración propia.

4.3.3. Árbol de potencia

El principal aprendizaje del hardware es que el diseño de potencia debe preceder al diseño mecánico y de software. En las primeras integraciones, gran parte de la energía de periféricos se entregaba desde la Raspberry Pi 5. Ese enfoque funciona para pruebas de bajo alcance, pero se vuelve frágil cuando se combinan SDR, LTE, GNSS, procesamiento sostenido y tráfico de red.

La evidencia de telemetría muestra tres hechos prácticos:

- Las pruebas con HackRF y peticiones desde servidor elevaron la temperatura del procesador hasta el orden de 75 °C.
- El riel EXT5V presentó caídas bajo cargas pesadas, aun cuando no todas las corridas terminaron marcando una bandera persistente de subtensión.

- Las pruebas con alimentación externa para periféricos tuvieron mejor margen para sostener carga combinada.

Por ello, el criterio para un nuevo producto o una siguiente versión debe ser:

El nodo debe tener un árbol de potencia externo y generoso, con capacidad para la carga actual y para extensiones futuras. La Raspberry Pi 5 debe ser tratada como carga y controlador, no como fuente primaria para el resto del sistema.

Tabla 4.2: Criterios mínimos para una versión robusta del árbol de potencia.

Criterio	Implicación de diseño
Fuente principal sobredimensionada	Debe alimentar simultáneamente Raspberry Pi 5, HackRF, Mon-RaF2, LTE/GNSS y periféricos futuros sin operar en el límite.
Rieles distribuidos	Las cargas de alto consumo o con transitorios, como SDR y LTE, deben alimentarse desde rieles dedicados o reguladores con margen, no desde el puerto de la Raspberry Pi.
Medición de rieles	El sistema debe conservar telemetría de voltaje, corriente, temperatura y eventos de subtensión para diagnóstico.
Protección y desacople	Se deben incluir protecciones, filtrado y capacitancia local cerca de cargas ruidosas o pulsantes.
Reserva para extensiones	El diseño debe contemplar nuevos servicios, antenas, sensores, almacenamiento o comunicaciones sin rediseñar la alimentación desde cero.

4.4. Entradas, salidas e interfaces

Tabla 4.3: Interfaces principales del hardware ANE2.

Interfaz	Tipo	Contrato operacional
Antenas MonRaF2	a RF coaxial	Las antenas se conectan a MonRaF2. La etapa RF de la tarjeta selecciona o acondiciona rutas antes de la adquisición o uso LTE/GNSS.
MonRaF2 HackRF	a RF seleccionado	El HackRF debe recibir una ruta RF controlada, no una conexión de antena asumida como directa.
HackRF Raspberry Pi 5	a USB	Transporta muestras IQ y comandos de configuración. El <i>sample rate</i> aumenta simultáneamente el ancho instantáneo y la presión sobre USB/CPU.
Raspberry Pi 5 a MonRaF2	Pi GPIO, UART, USB	Permite control auxiliar, comunicación con LTE/GNSS y señales de estado.
MonRaF2 red celular	a LTE	Provee conectividad de datos para estado, mediciones, campañas y reintentos.
MonRaF2 GNSS	a GNSS/GPS	Entrega ubicación del nodo para reportes y trazabilidad de mediciones.
Fuente externa a cargas	DC regulado	Debe distribuir potencia a Raspberry Pi 5, HackRF, MonRaF2 y expansiones con margen suficiente.

4.5. Decisiones de diseño o implementación

Tabla 4.4: Decisiones de diseño de hardware y justificación.

Decisión	Justificación	Implicación práctica
Usar MonRaF2 como concentrador RF y de conectividad	La tarjeta contiene rutas de antena, filtros, mux/switch RF, LTE/GNSS y señales de control.	Las antenas se documentan y cablean hacia MonRaF2; el HackRF queda como receptor SDR dentro de la cadena, no como punto directo de antena.
Usar Raspberry Pi 5 como computador de borde	Tiene capacidad suficiente para orquestación, telemetría y control del SDR.	Debe protegerse de sobrecarga eléctrica; no debe alimentar todos los periféricos en una versión robusta.

Decisión	Justificación	Implicación práctica
Usar HackRF One como receptor SDR	Permite recepción flexible por software y adquisición IQ.	Requiere ganancia configurada con criterio y alimentación estable.
Separar potencia de control	Los problemas de sub-tensión aparecen cuando la distribución de energía queda demasiado acoplada a la Raspberry Pi.	El árbol de potencia externo pasa a ser requisito de producto.
Mantener telemetría de salud	Temperatura, voltajes, corrientes y banderas de <i>throttling</i> permiten diagnosticar fallas reales.	Cada prueba de RF o conectividad debe acompañarse de telemetría eléctrica.

4.6. Problemas presentados y ajustes realizados

Tabla 4.5: Problemas presentados, decisiones y ajustes realizados en hardware.

Problema	Evidencia	Causa probable	Decisión o ajuste	Resultado
Subtensión y bajo margen de potencia	Caídas de EXT5V bajo cargas SDR/LTE y escenarios con eventos de alimentación insuficiente.	Demasiadas cargas alimentadas o condicionadas desde la Raspberry Pi 5.	Diseñar un árbol de potencia externo, generoso y medible.	Requisito obligatorio para una versión robusta.
Aumento de temperatura bajo carga	Pruebas con HackRF y peticiones desde servidor alcanzaron temperaturas cercanas a 75 °C.	Procesamiento sostenido, actividad USB y carga RF/LTE simultánea.	Mantener disipación, ventilación y telemetría de temperatura.	La temperatura queda como restricción operativa.
Ambigüedad en ruta de antenas	El diseño incluye redes MAIN/GNSS/AUX, filtros y mux/switch RF en MonRaF2.	Interpretar la antena como conexión directa al HackRF simplifica de forma incorrecta la cadena RF.	Documentar MonRaF2 como entrada y selección RF.	La conexión física queda alineada con el esquemático.
Dependencia del enlace LTE	Las pruebas de campo usan conectividad celular para operación y envío de datos.	La red celular puede variar por cobertura, antena, SIM y ubicación.	Separar diagnóstico de RF, cómputo, potencia y LTE.	Las fallas se pueden atribuir con mejor evidencia.
Riesgo de saturación RF	HackRF no usa AGC de hardware en esta cadena de operación.	Ganancia excesiva o emisores fuertes pueden saturar el ADC de 8 bits.	Ajustar ganancia por escenario y conservar configuraciones durante comparaciones.	Menor riesgo de confundir artefactos con emisiones reales.

4.7. Validación, pruebas o evidencias

La validación usada para esta sección combina inspección de esquemático, fotografías del montaje y telemetría de nodos bajo reposo, carga SDR, carga LTE y carga combinada. Los indicadores más útiles fueron temperatura ARM, EXT5V, corrientes de rieles, banderas de subtensión, frecuencia de CPU y estados de *throttling*.

Tabla 4.6: Resumen de telemetría eléctrica y térmica inspeccionada.

Escenario	Temperatura ARM	EXT5V	UV	Lectura para operación
Nodo en reposo	54.3–57.6 °C	5.006–5.041 V	No	Referencia base con sistema operativo y telemetría.
HackRF en carga alta	54.9–74.7 °C	4.748–4.965 V	No	La carga SDR aumenta demanda y temperatura.
Periféricos conectados en reposo	45.0–52.1 °C	4.903–4.963 V	No	El margen mejora cuando no hay procesamiento pesado.
Petición media con periféricos	61.5–75.7 °C	4.721–5.083 V	No	El sistema se acerca a límites térmicos y de alimentación.
Carga alta con periféricos externos	49.4–75.2 °C	4.904–5.122 V	No	La alimentación externa mejora el margen frente a carga combinada.

4.8. Aprendizajes y transferencia de conocimiento

- El árbol de potencia define la robustez del nodo. Si se alimentan periféricos exigentes desde la Raspberry Pi, el diseño queda sin margen.
- Las antenas pertenecen a la cadena de MonRaF2 y su etapa RF. El HackRF debe verse como receptor dentro de una ruta seleccionada, no como destino directo de todas las antenas.
- La telemetría eléctrica debe acompañar cualquier prueba RF. Sin voltajes, corrientes y temperatura es fácil confundir fallas de potencia con fallas de software.
- El diseño debe reservar margen para servicios futuros. Cada extensión de software o hardware aumenta demanda de energía, temperatura y tráfico.

4.9. Limitaciones

- Las cifras de telemetría resumen pruebas disponibles; no reemplazan una caracterización eléctrica completa con instrumentos de laboratorio.
- No se presentan curvas completas de consumo por banda, ganancia, temperatura ambiente, antena o ubicación.
- El diseño RF requiere validar pérdidas, aislamiento, adaptación de impedancias y compatibilidad electromagnética antes de una versión final de campo.
- La estabilidad LTE/GNSS depende de cobertura, antenas, SIM, ubicación y configuración del operador.

4.10. Recomendaciones específicas

1. Diseñar una fuente principal externa con margen amplio para carga simultánea y extensiones futuras.
2. Alimentar HackRF, LTE/GNSS y cargas auxiliares desde rieles dedicados o reguladores adecuados, no desde el margen de la Raspberry Pi 5.
3. Mantener medición de voltaje, corriente, temperatura y banderas de subtensión como parte del producto.
4. Conectar y documentar antenas desde MonRaF2 y su ruta tipo mux/switch RF.
5. Revisar disipación térmica para escenarios con SDR y LTE activos al mismo tiempo.
6. Separar pruebas de reposo, SDR, LTE/GNSS y carga combinada para poder atribuir fallas con evidencia.

Capítulo 5

Software del sensor

5.1. Resumen

Este documento describe, de forma pedagogica y conceptual, la arquitectura del código que corre directamente en el sensor de monitoreo espectral. El sensor está compuesto por una Raspberry Pi 5, una HackRF One y una tarjeta auxiliar que integra comunicación LTE, receptor GPS y un multiplexor para selección de antena.

La sección del proyecto descrita aquí no corresponde a toda la plataforma. El sistema completo también incluye componentes externos, como backend, frontend, bases de datos y servicios de visualización. Este repositorio se concentra en el código embebido del sensor: la parte que recibe instrucciones de la plataforma, controla el hardware de radiofrecuencia, adquiere muestras, procesa señales y devuelve resultados.

5.1.1. Propósito general

El propósito general del código es convertir al sensor físico en un nodo autónomo de medición espectral. Para ello, el software debe recibir órdenes del servidor, configurar la HackRF One, seleccionar la antena adecuada, capturar señales IQ, procesarlas localmente y enviar a la plataforma central productos útiles como PSD, métricas de modulación, audio demodulado, estado del dispositivo y coordenadas GPS.

5.1.2. Objetivos específicos

- Controlar la adquisición de radio mediante la HackRF One, ajustando frecuencia central, tasa de muestreo, ganancias, amplificador y puerto de antena.
- Ejecutar mediciones espectrales en modo *realtime* y en campañas programadas desde

el servidor.

- Procesar muestras IQ crudas para estimar densidad espectral de potencia mediante metodos como Welch y banco de filtros polifase.
- Demodular senales AM o FM cuando el servidor solicita audio en tiempo real.
- Aplicar etapas de limpieza y correccion para mejorar la calidad de las mediciones, incluyendo balance IQ y remocion del pico DC.
- Mantener una calibracion de frecuencia que compense desviaciones del oscilador de la HackRF.
- Coordinar la comunicacion con el backend, el envio de resultados, los reintentos ante fallos de red y el reporte de estado del sensor.
- Gestionar recursos fisicos del sensor, como LTE, GPS, GPIO y multiplexor de antena.

5.2. Introduccion y arquitectura general

La arquitectura del sensor se puede entender como una cadena de trabajo: el servidor decide que se debe medir, el orquestador traduce esa instruccion a una configuracion de radio, el motor RF adquiere y procesa las muestras, y finalmente los resultados regresan al servidor.

La division principal del sistema es la siguiente:

- **Orquestacion en Python:** gestiona la relacion con el backend, consulta instrucciones, programa campanas, controla estados del sistema, envia comandos al motor RF y prepara los resultados para publicarlos.
- **Motor RF en C:** controla la HackRF One, recibe muestras IQ, administra buffers de alta velocidad, ejecuta procesamiento DSP, calcula PSD, demodula audio y devuelve resultados al orquestador.
- **Servicios auxiliares:** atienden funciones de soporte como GPS, LTE, estado del sistema, reintentos de datos no enviados, WebRTC para audio y control del multiplexor de antena.

La decision de separar Python y C responde a una necesidad practica. Python es adecuado para coordinacion, comunicacion HTTP, manejo de estados y programacion de tareas. C es adecuado para la parte sensible a latencia: captura de muestras, buffers, FFT, filtrado, PSD y demodulacion.

5.2.1. Servicios y modos de operacion

El sensor opera con varios servicios o modos complementarios:

- **PSD en tiempo real:** el backend solicita una configuracion y el sensor responde con una estimacion espectral de la banda capturada.
- **PSD y audio en tiempo real:** ademas de la PSD, el sensor demodula FM o AM y genera tramas de audio reproducibles por la plataforma.
- **Campanas:** el backend programa mediciones periodicas; el sensor guarda la configuracion, agenda ejecuciones y sube resultados en cada intervalo.
- **Calibracion en frecuencia:** el sensor estima el error de sintonia y aplica una correccion para que las mediciones queden mejor alineadas en frecuencia.
- **GPS y LTE:** el sensor mantiene conectividad celular y reporta ubicacion geografica al servidor.
- **Estado del dispositivo:** se reportan variables de salud como conectividad, almacenamiento, temperatura, memoria y actividad del software.

5.3. Bloques o partes del codigo

El codigo puede leerse como un conjunto de bloques funcionales. Cada bloque cumple una responsabilidad dentro del camino completo de la senal: desde la instruccion del backend hasta la publicacion del resultado.

5.3.1. Adquisicion

El bloque de adquisicion es el encargado de hablar con la HackRF One y convertir una instruccion de medicion en muestras IQ reales.

Conceptualmente, este bloque realiza cinco acciones:

1. Recibe una configuracion de medicion enviada por el orquestador. Esa configuracion contiene parametros como frecuencia central, ancho de muestreo, ganancias, metodo de PSD, puerto de antena, filtro solicitado y modo de demodulacion.
2. Aplica la configuracion sobre la HackRF One. Esto incluye sintonia de frecuencia, tasa de muestreo, ganancias de recepcion y correccion de frecuencia cuando existe una calibracion disponible.

3. Inicia la recepcion de muestras IQ crudas desde el dispositivo SDR.
4. Deposita esas muestras en buffers circulares para separar la velocidad de llegada del hardware de la velocidad de procesamiento del software.
5. Entrega bloques de muestras suficientemente grandes a los algoritmos de procesamiento, evitando que cada operacion pesada interrumpa directamente la captura.

Las muestras IQ representan la senal recibida en banda base compleja. La componente I corresponde a la parte en fase y la componente Q a la parte en cuadratura. En conjunto permiten conservar informacion de amplitud y fase, que es necesaria para estimar el espectro, filtrar, demodular FM o demodular AM.

5.4. Uso de buffers

La logica de buffers es central en el sensor. La HackRF entrega datos de forma continua y rapida, mientras que la PSD, el filtrado o la demodulacion requieren procesamiento por bloques. Por eso el motor RF usa buffers circulares:

- Un buffer principal recibe muestras IQ para estimacion espectral.
- Un buffer de audio conserva muestras necesarias cuando se activa demodulacion AM o FM.
- El software drena estos buffers cuando ya hay suficientes muestras para construir una respuesta.

Esta estructura permite que el sensor atienda procesos en tiempo real, como PSD y audio, sin tratar cada muestra de manera aislada. La idea pedagogica es similar a una banda transportadora: el hardware deposita muestras, el procesamiento toma paquetes completos y el orquestador recibe resultados ya elaborados.

5.5. Relacion con el backend

La adquisicion no decide por si sola que medir. Las instrucciones provienen del backend y llegan al motor RF a traves del orquestador. El backend define la intencion; el orquestador valida y traduce esa intencion; el motor RF la ejecuta sobre el hardware.

Este esquema permite que el sensor funcione como un nodo remoto controlado por la plataforma, pero conservando autonomia local para manejar errores, tiempos de espera, buffers, reconexiones y estados internos.

5.5.1. Preproceso

El preproceso agrupa las operaciones que preparan la señal antes de extraer productos finales. Su objetivo es reducir artefactos propios del hardware y mejorar la calidad de la medición.

5.6. Balance IQ

En un receptor SDR real, las ramas I y Q no siempre quedan perfectamente balanceadas. Puede haber diferencias de ganancia, pequeños desplazamientos DC o correlación no deseada entre ambas componentes. Si estos efectos no se corrigen, pueden aparecer artefactos en el espectro o degradación en la demodulación.

El balance IQ busca compensar esos errores antes del cálculo de PSD o del filtrado. De forma conceptual, el algoritmo:

- remueve desplazamientos promedio en las ramas I y Q;
- compara la energía relativa de ambas ramas;
- corrige diferencias de amplitud;
- reduce correlaciones espurias entre I y Q.

No se trata de cambiar la información de la señal recibida, sino de disminuir imperfecciones introducidas por la cadena de recepción.

5.7. Corrección de componente DC

La componente DC aparece cerca del centro de la banda base y suele estar asociada a efectos internos del receptor. En el flujo del sensor se atiende en dos niveles:

- En el preproceso IQ se reduce el offset de las muestras antes de calcular PSD o demodular.
- En el postproceso de PSD se repara el pico central visible en el espectro cuando aparece como un artefacto de medición.

Esta separación es importante: una cosa es limpiar la señal cruda y otra es pulir el producto espectral que se enviara al servidor.

5.7.1. Proceso

El bloque de proceso transforma muestras IQ en productos utiles para la plataforma. En este repositorio se implementan tres familias principales de procesamiento: estimacion de PSD, demodulacion de audio y filtrado de senales.

5.8. Estimacion de PSD

La PSD, o densidad espectral de potencia, permite observar como se distribuye la energia de la senal dentro de un rango de frecuencias. Es el producto principal para monitoreo espectral, porque permite identificar ocupacion de banda, niveles relativos de potencia, portadoras y posibles emisiones.

El sensor contempla metodos como:

- **Welch**: divide la senal en segmentos, aplica ventanas, calcula FFT y promedia resultados para obtener una estimacion estable del espectro.
- **Banco de filtros polifase**: mejora la separacion entre canales y ayuda a controlar fugas espectrales cuando se requiere una estimacion mas selectiva.

La PSD es una de las tareas mas costosas del sistema, porque requiere operar sobre grandes bloques de muestras y ejecutar transformadas de Fourier. Por eso el motor RF esta escrito en C y aprovecha una logica de trabajo multihilo y bibliotecas de procesamiento eficientes.

5.9. Demodulacion FM y AM

Cuando el backend solicita audio en tiempo real, el sensor no solo calcula PSD. Tambien toma las muestras IQ, extrae la informacion modulada y la convierte en audio reproducible.

En FM, la informacion esta principalmente en la variacion de fase o frecuencia instantanea. El proceso conceptual consiste en observar como cambia la fase entre muestras consecutivas, convertir esa variacion en una senal de audio y acondicionar el resultado para reproduccion.

En AM, la informacion esta principalmente en la envolvente de la senal. El proceso conceptual consiste en calcular la magnitud de la senal compleja, separar las variaciones utiles de la portadora y normalizar el audio resultante.

Despues de demodular, el audio se convierte a un formato de transmision. El sensor genera tramas de audio, las codifica con Opus y las entrega al servicio local que publica el flujo mediante WebRTC. Esto permite que la plataforma escuche la senal con baja latencia sin que el backend tenga que procesar IQ crudo.

5.10. Filtrado

El filtrado permite concentrarse en una parte de la banda capturada y rechazar contenido fuera del rango de interes. El servidor puede solicitar un rango de frecuencia y el motor RF aplica un filtro pasabanda antes de calcular la PSD o de preparar la senal.

Desde una perspectiva conceptual, el filtro funciona como una ventana selectiva: conserva la region que interesa y atenúa lo que queda por fuera. En el codigo se apoya en procedimientos de dominio frecuencial y validaciones para que el rango solicitado sea coherente con la banda realmente capturada por la HackRF.

5.11. Trabajo multihilo

El sistema separa tareas para mantener capacidad de respuesta:

- Un flujo atiende la llegada de muestras desde la HackRF.
- Otro flujo procesa bloques para PSD.
- Cuando hay audio, un hilo adicional drena muestras, demodula y envia tramas codificadas.
- El orquestador permanece disponible para comunicarse con el backend y manejar cambios de modo.

Esta organizacion evita que una tarea pesada, como una PSD de alta resolucion, bloquee completamente la captura o la transmision de audio.

5.11.1. Postproceso

El postproceso es la pulida final aplicada a los resultados antes de enviarlos al servidor. En este sistema, la etapa mas importante es la correccion del pico DC en la PSD.

5.12. Remocion del pico DC en PSD

Aunque el preproceso reduce offset en las muestras IQ, en la PSD puede quedar un pico artificial cerca del centro de la banda. Ese pico no siempre representa una emision real; con frecuencia es un artefacto propio del receptor SDR.

El algoritmo de postproceso analiza el comportamiento estadístico de la PSD como una señal con variabilidad local. En lugar de borrar siempre un número fijo de puntos, estima la región afectada y decide cómo reconstruirla.

Conceptualmente, el procedimiento realiza:

- detección de la región central afectada usando simetría y cambios locales en la pendiente;
- análisis de ocupación espectral alrededor del centro;
- validaciones con descriptores estadísticos como media, mediana, dispersión y comportamiento de la cola alta;
- ampliación de la ventana de reparación cuando la zona tiene baja ocupación espectral;
- reconstrucción local mediante interpolación lineal o ajuste polinomial.

El resultado enviado al backend conserva una PSD más limpia. Cuando aplica, el software también puede conservar información de diagnóstico sobre la corrección, como la ventana reparada y el modo de reconstrucción usado.

5.13. Calibración en frecuencia

La calibración en frecuencia busca compensar la desviación entre la frecuencia nominal solicitada y la frecuencia real a la que queda sintonizado el receptor. En SDR de bajo costo, esta desviación puede aparecer por tolerancias del oscilador, temperatura o condiciones de operación.

5.13.1. Idea principal

El sensor usa una referencia conocida para estimar el error de frecuencia. En el caso de emisiones FM comerciales, una referencia útil es el tono piloto estereo de 19 kHz que aparece después de demodular FM. Si una estación FM fuerte contiene ese tono, el sensor puede usarlo como punto de comparación.

El flujo conceptual es:

1. Capturar una porción amplia de la banda donde se esperan emisoras FM.
2. Identificar candidatos con potencia suficiente.

3. Bajar cada candidato a banda base y demodular FM.
4. Buscar el tono piloto alrededor de 19 kHz en la señal demodulada.
5. Comparar la posición observada con la posición esperada.
6. Estimar un error de frecuencia y expresarlo como factor de corrección.

5.13.2. Uso de varias emisiones

Cuando hay varias emisiones candidatas, el sistema puede usar más de una referencia para evitar depender de una sola estación. Las emisiones más confiables reciben mayor peso, por ejemplo si tienen mejor potencia o una detección más clara del tono piloto. Con esa información se obtiene una estimación más robusta del error.

5.13.3. Aplicación de la corrección

El resultado de la calibración se guarda como un error de frecuencia, normalmente expresado en partes por millón. Luego, cuando el backend solicita una frecuencia central, el orquestador incluye ese factor en la configuración enviada al motor RF.

El motor usa la corrección para ajustar la sintonía de la HackRF. De esta manera, la frecuencia física de adquisición queda más alineada con la frecuencia nominal que la plataforma espera visualizar.

5.14. Orquestador

El orquestador es la lógica de coordinación del sensor. Su función no es procesar IQ de forma intensiva, sino dirigir el trabajo de todos los bloques.

5.14.1. Responsabilidades principales

- Consultar al backend para saber si hay solicitudes en tiempo real.
- Consultar y sincronizar campanas programadas.
- Validar configuraciones antes de enviarlas al motor RF.
- Enviar comandos al motor RF usando mensajería local ZeroMQ.
- Recibir resultados de PSD y métricas desde el motor RF.

- Aplicar postproceso sobre los resultados cuando corresponde.
- Formatear y publicar datos hacia el backend mediante HTTP REST.
- Controlar el inicio y parada del servicio de audio WebRTC.
- Mantener estados globales como `IDLE`, `REALTIME`, `CAMPAIGN`, `KALIBRATING` y `ERROR`.
- Gestionar memoria compartida local para parametros persistentes, como `ppm_error`, coordenadas GPS o configuraciones de campana.

5.14.2. Modos controlados por el orquestador

5.15. Campanas

Las campanas son mediciones programadas por el servidor. El orquestador consulta las campanas activas, selecciona la que corresponde ejecutar, guarda sus parametros y agenda ejecuciones periodicas. Cada ejecucion captura PSD y envia el resultado al backend. Si la red falla, el resultado puede quedar en una cola local para ser reenviado posteriormente.

5.16. PSD en tiempo real

En modo realtime, el servidor entrega una configuracion inmediata. El orquestador la envia al motor RF, espera la respuesta, aplica postproceso y sube el resultado. Mientras el backend siga enviando una solicitud valida, el sensor mantiene este modo activo.

5.17. PSD y audio en tiempo real

Cuando la solicitud incluye demodulacion, el orquestador activa tambien el camino de audio. En ese caso, el motor RF demodula AM o FM y genera tramas Opus; el servicio WebRTC se encarga de hacer disponible ese audio para la plataforma.

5.18. Calibracion

Antes o durante ciertos ciclos de operacion, el orquestador puede solicitar al motor RF una calibracion de frecuencia. El resultado se guarda localmente y se usa en adquisiciones posteriores.

5.19. Protocolos y procedimientos nombrados

El documento evita entrar en detalles de bajo nivel, pero es importante reconocer los protocolos y procedimientos principales que conectan los bloques:

Elemento	Uso conceptual en el sensor
HTTP REST	Comunicacion entre sensor y backend para consultar instrucciones y publicar datos.
ZeroMQ	Canal local de comandos entre el orquestador Python y el motor RF en C.
libhackrf/USB	Interfaz de control y adquisicion de muestras desde la HackRF One.
WebRTC	Entrega de audio demodulado hacia cliente/plataforma con baja latencia.
Opus	Codificacion de audio antes de enviarlo al servicio WebRTC.
GPIO	Control de lineas fisicas para multiplexor de antena y soporte de hardware.
PPP/LTE	Conectividad celular del sensor hacia la red.
GPS/NMEA	Obtencion y conversion de coordenadas geograficas.
Welch	Metodo de estimacion de PSD por segmentos y promediado.
Banco polifase	Metodo de estimacion espectral con mejor control de separacion entre canales.

5.20. Flujo completo de funcionamiento

De forma resumida, el funcionamiento del sensor puede leerse como el siguiente ciclo:

1. El sensor se identifica ante la plataforma mediante su direccion MAC y su estado local.
2. El orquestador consulta si hay una tarea realtime o una campana activa.
3. Si hay tarea, se construye una configuracion de radio y se envia al motor RF.
4. El motor RF configura la HackRF, selecciona antena si aplica y captura IQ.
5. Las muestras pasan por preproceso para mejorar su condicion de medicion.

6. El bloque de proceso calcula PSD, aplica filtros y, si se solicita, demodula audio.
7. El postproceso corrige artefactos finales, especialmente el pico DC en la PSD.
8. El orquestador empaqueta el resultado y lo envia al backend.
9. Si el envio falla, el dato puede quedar almacenado localmente para reintento.
10. Los servicios auxiliares mantienen GPS, LTE, estado del sensor y audio segun el modo activo.

Capítulo 6

Plataforma web en la nube

6.1. Propósito del capítulo

Este capítulo documenta la plataforma web que constituye la capa de interfaz humana y de coordinación central del sistema de monitoreo espectral ANE. La plataforma está compuesta por un frontend de aplicación de página única (SPA) y un backend de servicios que actúa como centro de orquestación entre las personas operadoras, los sensores RF en campo, la base de datos y los servicios de postprocesamiento.

El capítulo describe la arquitectura general, las funcionalidades ofrecidas a los usuarios, las interfaces de comunicación entre componentes, las decisiones de diseño que guiaron la implementación, los problemas encontrados durante el desarrollo, los aprendizajes obtenidos y las limitaciones actuales. El objetivo es proporcionar una comprensión completa de cómo la plataforma web hace posible el ciclo operativo completo: desde la autenticación de una persona usuaria hasta la generación de un reporte de cumplimiento normativo basado en mediciones de espectro capturadas en campo.

6.2. Contexto dentro del sistema general

La plataforma web ocupa la posición central en la arquitectura del sistema ANE. Como se describe en el capítulo de diseño general (Capítulo 3), el sistema se organiza en tres dominios principales: borde (sensores RF y adquisición), nube (plataforma web, base de datos y postprocesamiento) y usuarios (personas operadoras y administradoras). La plataforma web constituye la totalidad del dominio de nube desde la perspectiva de coordinación y la totalidad del dominio de usuarios desde la perspectiva de interfaz.

La relación con los demás módulos del sistema es la siguiente:

- **Hardware y sensores RF (Capítulo 4):** la plataforma recibe los datos de espectro, estado, GPS y audio demodulado que los sensores capturan en campo. También entrega a los sensores las configuraciones de adquisición y las campañas programadas que deben ejecutar.
- **SDR y adquisición en borde (Capítulo 5):** el flujo de adquisición definido en el borde depende de los parámetros que la plataforma configura y transmite. La plataforma es la consumidora de los datos generados por la cadena de procesamiento SDR.
- **Análisis espectral y localización en la nube (Capítulo 7):** la plataforma invoca al servicio de postprocesamiento Python para el análisis de emisiones, detección de señales, cruce con licencias y evaluación de cumplimiento normativo. Los resultados del análisis se integran en los reportes que la plataforma entrega a las personas usuarias.
- **Protocolos de prueba (Capítulo 8):** la plataforma es el objeto de pruebas de integración extremo a extremo y pruebas de usabilidad. Los protocolos de prueba definidos en ese capítulo se aplican sobre la plataforma para verificar su correcto funcionamiento en conjunto con los demás módulos.
- **Integración general (Capítulo 9):** la plataforma es el componente integrador por excelencia. El capítulo de integración detalla cómo todos los módulos se conectan a través de la plataforma para formar el sistema completo.

En el flujo extremo a extremo del sistema, la plataforma web es el punto de entrada para las personas y el punto de convergencia para los datos. Sin la plataforma, los sensores podrían capturar datos pero no habría forma de configurarlos, visualizar sus mediciones, planificar campañas, detectar alertas ni generar reportes.

6.3. Desarrollo técnico

La plataforma web se implementa como dos aplicaciones independientes que se despliegan como contenedores Docker coordinados por un proxy inverso Nginx: un frontend de aplicación de página única (SPA) y un backend de servicios API.

6.3.1. Frontend

El frontend es una aplicación web de página única desarrollada con React 19 y TypeScript, construida con el empaquetador Vite. La interfaz sigue un diseño de panel de control con menú lateral de navegación que da acceso a siete pantallas principales.

Tecnologías

La pila tecnológica del frontend está conformada por:

- **React 19** como librería principal de componentes de interfaz, seleccionado por su modelo eficiente de actualización del DOM virtual y su capacidad para manejar datos en tiempo real sin re-renderizaciones innecesarias.
- **TypeScript** como lenguaje de desarrollo, proporcionando tipado estático que mejora la detección temprana de errores y documenta implícitamente las interfaces de datos entre componentes.
- **Vite 5** como empaquetador y servidor de desarrollo, seleccionado por su velocidad de compilación basada en ESM nativo y su configuración simplificada.
- **Tailwind CSS 3** como framework de estilos mediante utilidades atómicas, facilitando la iteración rápida sobre el diseño y la consistencia visual entre componentes.
- **Plotly.js** para la visualización interactiva de espectro de potencia y gráficas waterfall (espectrograma), seleccionada por su soporte nativo para gráficos científicos con miles de puntos de datos y actualización en tiempo real.
- **Leaflet** y **React-Leaflet** para la visualización de mapas geográficos con marcadores de sensores, permitiendo la ubicación espacial de la red de monitoreo.
- **Recharts** para gráficas estadísticas complementarias en el panel de inicio.
- **React Router 7** para el enrutamiento del lado del cliente, permitiendo navegación entre pantallas sin recarga completa de la página.
- **Microsoft Authentication Library (MSAL)** para la integración con Azure AD como proveedor de identidad institucional.
- **Axios** como cliente HTTP para la comunicación con el backend.

Pantallas principales

La interfaz se organiza en las siguientes pantallas, accesibles desde un menú lateral cuya visibilidad se adapta al rol de la persona usuaria:

Inicio (Dashboard): pantalla de bienvenida que presenta un mapa geográfico con la ubicación de todos los sensores registrados, indicadores de estado por colores, y un resumen de estadísticas del sistema (sensores activos, campañas en curso, alertas recientes). Es el punto de partida para la navegación.

Dispositivos: vista de gestión de la red de sensores. Muestra el listado completo de sensores con su estado actual, ubicación y antenas asociadas. Incluye un mapa interactivo con marcadores de estado y acceso al detalle de cada sensor. Desde esta pantalla se puede iniciar un monitoreo en vivo sobre un sensor seleccionado.

Monitoreo en vivo: flujo central de la plataforma para realizar mediciones inmediatas. La persona selecciona un sensor y una antena, define los parámetros de adquisición (rango de frecuencia, ancho de banda, resolución, ganancias) e inicia la captura. Durante la medición se visualiza en tiempo real el espectro de potencia (PSD) mediante Plotly.js, la evolución temporal en waterfall, y el audio demodulado AM/FM cuando está habilitado. La persona puede detener la medición o guardar la configuración como una nueva campaña.

Campañas: gestión de mediciones programadas. Permite crear campañas definiendo nombre, fechas de inicio y fin, horas de operación, intervalo entre mediciones, parámetros espectrales y sensores asignados. Incluye listado con filtros por estado, visualización de detalle, inicio y detención manual, y consulta de datos capturados. Los datos de campaña son la fuente para la generación de reportes de cumplimiento.

Alertas: revisión de incidentes y eventos operativos detectados por el sistema. Presenta un listado cronológico con filtros por sensor, tipo de alerta y rango de fechas. Cada alerta se vincula con los datos del sensor para facilitar el diagnóstico.

Configuración: panel administrativo para la gestión de recursos. Incluye CRUD del catálogo de antenas (tipo, rango de frecuencia, ganancia, código de inventario), administración de cuentas de usuario (roles y permisos), y configuración de parámetros globales del sistema.

Audio: reproductor de audio demodulado AM/FM, accesible desde el monitoreo en vivo o desde la vista de un sensor. Muestra métricas de modulación en tiempo real: profundidad para AM, excursión para FM.

Mecanismo de actualización en tiempo real

El frontend mantiene una conexión WebSocket con el backend para recibir actualizaciones sin necesidad de recarga manual. Los eventos recibidos incluyen cambios de estado de sensores, nuevas ubicaciones GPS, datos de espectro en tiempo real y confirmaciones de comandos. Estos eventos actualizan automáticamente los mapas, gráficas e indicadores en todas las pantallas relevantes, implementando un modelo de datos reactivo donde la interfaz refleja el estado del sistema en todo momento.

6.3.2. Backend

El backend es el centro de coordinación de la plataforma, implementado como un servidor Node.js con TypeScript sobre el framework Express. Su función principal es conectar a

las personas usuarias (a través del frontend), los sensores RF en campo, la base de datos PostgreSQL y los servicios de postprocesamiento.

Tecnologías

- **Node.js 18+** como entorno de ejecución, seleccionado por su modelo de E/S no bloqueante que permite manejar múltiples conexiones concurrentes (frontend, sensores, WebSocket) en un solo proceso.
- **TypeScript** como lenguaje, compilado a JavaScript para producción, proporcionando verificación de tipos en tiempo de compilación.
- **Express** como framework HTTP, ofreciendo un modelo maduro de middleware, enrutamiento y manejo de solicitudes.
- **WebSocket nativo** (librería ws) para comunicación en tiempo real con el frontend, seleccionado sobre alternativas como Socket.IO por su menor sobrecarga de protocolo.
- **PostgreSQL 13+** con cliente pg y pool de conexiones, como sistema de base de datos relacional.
- **TimescaleDB** como extensión opcional de PostgreSQL para optimización de datos de series temporales.
- **JSON Web Tokens (JWT)** y librería jose para la gestión de sesiones y autenticación.
- **Swagger/OpenAPI** para la documentación interactiva de la API.

Capas de la arquitectura

El backend está organizado en seis capas lógicas con responsabilidades bien definidas:

Capa de entrada HTTP: expone el servidor en el puerto configurado (3000) y aplica middleware común: CORS para permitir solicitudes del frontend, registro (logging) de cada solicitud con método y ruta, y parsing de JSON con un límite ampliado de 50 MB para permitir la recepción de tramas espectrales de alta resolución enviadas por los sensores.

Capa de autenticación y autorización: valida la identidad de la persona usuaria mediante dos mecanismos complementarios. Para el acceso institucional desde el frontend, se integra con Azure AD utilizando el protocolo OpenID Connect y la librería jose para validación de tokens. Para el acceso administrativo local, se utilizan credenciales almacenadas en la base de datos con hash bcrypt y tokens JWT firmados por el servidor. Los middleware de autenticación se aplican de forma selectiva según la ruta: los endpoints de

ingesta de datos desde sensores operan con validación por dirección MAC; los endpoints de administración requieren sesión validada con rol apropiado.

Capa de rutas y controladores: organiza la lógica de negocio en módulos independientes: autenticación, gestión de sensores y antenas (CRUD, asignación), recepción de datos de campo (estado, GPS, espectro, audio), control remoto de sensores, campañas de medición, reportes de cumplimiento y configuración del sistema.

Capa de modelos y acceso a datos: abstrae las consultas SQL mediante funciones tipadas que utilizan un pool de conexiones pg.Pool con hasta 20 conexiones simultáneas. Las operaciones que modifican múltiples tablas se ejecutan dentro de transacciones PostgreSQL para garantizar consistencia.

Capa de comunicación en tiempo real: mantiene un servidor WebSocket que transmite cinco tipos de eventos al frontend: **sensor_status** (estados de sensores), **sensor_gps** (ubicaciones), **sensor_data** (espectro en tiempo real), **sensor_configure** y **sensor_stop** (comandos). Un segundo servidor WebSocket independiente gestiona el streaming de audio demodulado en formato Opus.

Capa de mantenimiento automático: ejecuta tareas programadas en intervalos regulares de 30 segundos. La tarea principal valida la antigüedad del último reporte de cada sensor activo y marca como offline aquellos que exceden el umbral de inactividad. Los cambios de estado se notifican al frontend mediante broadcast WebSocket.

API REST

La interfaz de programación del backend expone aproximadamente treinta endpoints organizados en ocho categorías funcionales, todas documentadas mediante especificación OpenAPI y accesibles a través de Swagger UI:

1. **Sensor Data:** endpoints POST para que los sensores envíen estado operativo, ubicación GPS, datos de espectro radioeléctrico y audio demodulado. Son los endpoints de mayor tráfico durante la operación normal.
2. **Sensor Query:** endpoints GET para consulta de datos históricos: último estado, última ubicación, últimos datos de espectro, datos por rango temporal y configuración activa, identificando cada sensor por su dirección MAC.
3. **Sensor Control:** endpoints POST para enviar comandos a los sensores: configuración de parámetros de escaneo, comando de detención y almacenamiento de configuración activa.
4. **Sensors Management:** CRUD completo de sensores más consultas por ID o MAC, y gestión de la relación sensor-antena (asignar, desasignar, consultar).
5. **Antennas Management:** CRUD completo del catálogo de antenas.

6. **Campaigns:** gestión de campañas con operaciones de creación, consulta, actualización, eliminación, inicio, detención y consulta de datos capturados.
7. **Reports:** generación de reportes de cumplimiento normativo que integran datos de campaña, ubicación y análisis del servicio de postprocesamiento.
8. **System:** endpoint raíz con versión de la API y directorio de endpoints.

6.3.3. Base de datos

La plataforma utiliza PostgreSQL como sistema de gestión de base de datos relacional. El esquema está organizado en cuatro dominios funcionales que agrupan quince tablas:

Dominio de identidad y acceso: almacena credenciales y roles de las personas usuarias (tabla `users`, con hash bcrypt para contraseñas) y un registro de auditoría de acciones sensibles (tabla `audit_logs`).

Dominio de sensores y configuración: mantiene el registro maestro de sensores RF identificados por MAC única (tabla `sensors`), el catálogo de antenas con sus parámetros técnicos (`antennas`), la relación muchos-a-muchos entre sensores y antenas con puerto de conexión (`sensor_antennas`), y los parámetros de adquisición por sensor: frecuencias, sample rate, ganancias, ventana, solapamiento y configuración de demodulación (`sensor_configurations`).

Dominio de datos de campo: almacena las series temporales generadas por los sensores: estado operativo con métricas de CPU, RAM, temperatura y latencia (`sensor_status`), ubicaciones GPS (`sensor_gps`), datos de espectro con arreglos de potencia, métricas de demodulación AM/FM y asociación opcional a campaña (`sensor_data`), y alertas operativas históricas (`sensor_history_alert`).

Dominio de campañas y reportes: define las campañas de medición con parámetros de programación y configuración espectral (`campaigns`), la asignación de sensores a campañas (`campaign_sensors`), el caché de reportes de cumplimiento en formato JSON (`compliance_reports_cache`), y parámetros globales del sistema en formato clave-valor (`system_configurations`).

La tabla `sensor_status`, que recibe actualizaciones periódicas de cada sensor y es la de mayor crecimiento, está configurada como hypertable de TimescaleDB, particionada por la columna `timestamp_ms`. Esta configuración permite compresión y retención automática de datos, y optimiza consultas temporales mediante índices compuestos por MAC y timestamp.

6.3.4. Despliegue

La plataforma se despliega en un servidor institucional privado de la ANE, accesible a través de la red interna de la entidad. La arquitectura de despliegue utiliza contenedores Docker coordinados por un proxy inverso Nginx:

- **Frontend:** imagen Docker construida en dos etapas: compilación con Node.js Alpine y servidor web con Nginx Alpine. Expone el puerto 80 con configuración para aplicaciones de página única (SPA) y compresión gzip.
- **Backend:** imagen Docker construida en dos etapas: compilación de TypeScript y ejecución con Node.js Alpine. Expone el puerto 3000.
- **Nginx:** actúa como punto único de entrada, enrutando el tráfico HTTP al frontend (archivos estáticos) y al backend (API REST en `/api/`, WebSocket en `/ws/`). La configuración incluye timeouts extendidos de una hora para la generación de reportes de larga duración.
- **PostgreSQL:** base de datos con extensión TimescaleDB, ejecutándose en el mismo servidor. En producción se recomienda la imagen `timescale/timescaledb:latest-pg14`.

La configuración de entorno se gestiona mediante variables de entorno (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `PORT`) que permiten desplegar la misma imagen en entornos de desarrollo, pruebas y producción sin recompilación.

6.4. Entradas, salidas e interfaces

6.4.1. Entradas

La plataforma recibe datos de tres fuentes principales:

Desde el frontend: solicitudes HTTP de las personas usuarias para autenticarse, consultar información, configurar mediciones, gestionar campañas, revisar alertas y generar reportes. Estas solicitudes incluyen tokens de autenticación (JWT o Azure AD) y parámetros específicos según la operación.

Desde los sensores RF: datos de campo en formato JSON enviados mediante solicitudes POST:

- Estado del sensor: métricas de CPU por núcleo, RAM, disco, temperatura, latencia de red y logs del sistema operativo del dispositivo.
- Ubicación GPS: latitud, longitud y altitud.

- Datos de espectro: arreglo de potencias PSD (P_{xx}), frecuencia inicial y final en Hz, marca de tiempo en epoch milliseconds, coordenadas de captura, y métricas de demodulación AM (profundidad) o FM (excursión).
- Audio demodulado: tramas de audio PCM codificadas en base64 o en formato Opus.

Desde el servicio de postprocesamiento: resultados de análisis de emisiones en formato JSON, con frecuencias centrales detectadas, anchos de banda, potencias, cruce con licencias y evaluación de cumplimiento por municipio (código DANE).

6.4.2. Salidas

La plataforma entrega:

Al frontend: respuestas JSON a todas las consultas y operaciones, eventos WebSocket para actualización en tiempo real de estado de sensores, GPS, datos de espectro y audio.

A los sensores RF: configuraciones de adquisición (frecuencias, sample rate, ganancias, ventana, solapamiento, demodulación) y listas de campañas asignadas con todos los parámetros necesarios para ejecutar mediciones programadas.

A las personas usuarias: reportes de cumplimiento normativo estructurados con emisiones detectadas, comparación contra licencias, evaluación por parámetro (frecuencia central, ancho de banda, potencia) y estimación de intensidad de campo.

6.4.3. Interfaces

Las interfaces de comunicación entre componentes son:

- **Frontend – Backend:** API REST sobre HTTP con formato JSON. Autenticación mediante tokens JWT en cabecera **Authorization**. WebSocket para eventos en tiempo real y streaming de audio.
- **Sensores – Backend:** API REST sobre HTTP con formato JSON. Identificación del sensor por dirección MAC. Los sensores actúan como clientes HTTP que envían datos (POST) y consultan configuración (GET).
- **Backend – PostgreSQL:** conexión TCP nativa mediante el cliente pg sobre el puerto 5432, utilizando un pool de conexiones configurable.
- **Backend – Postprocesamiento:** invocación HTTP síncrona al servicio Python Flask expuesto en el puerto 8000, con formato JSON para solicitudes y respuestas.

- **Nginx – Frontend:** entrega de archivos estáticos (HTML, JS, CSS, assets) por HTTP en puerto 80.
- **Nginx – Backend:** proxy inverso HTTP y WebSocket, con timeouts extendidos para operaciones de larga duración.

6.5. Decisiones de diseño o implementación

6.5.1. Separación frontend/backend con API REST

Se decidió separar el frontend y el backend como aplicaciones independientes comunicadas por API REST, en lugar de una aplicación monolítica con renderizado en el servidor. Esta decisión se fundamenta en tres consideraciones:

1. **Despliegue independiente:** el frontend (archivos estáticos) y el backend (servidor Node.js) pueden escalarse, actualizarse y desplegarse de forma independiente. El frontend se sirve desde Nginx como archivos estáticos con caché agresiva; el backend maneja la lógica de negocio y la concurrencia.
2. **Separación de responsabilidades:** el equipo de frontend puede iterar sobre la interfaz sin afectar la lógica del backend, y viceversa. La API actúa como contrato estable entre ambas capas.
3. **Cliente único vs. múltiples clientes:** aunque actualmente el frontend web es el único cliente, la arquitectura REST permite que en el futuro otros clientes (aplicaciones móviles, scripts de automatización, dashboards externos) consuman la misma API sin modificar el backend.

6.5.2. WebSocket sobre polling para tiempo real

La elección de WebSocket como mecanismo de comunicación en tiempo real, en lugar de polling HTTP periódico, se justifica por el perfil de tráfico del sistema:

- Durante un monitoreo activo, un sensor puede enviar datos de espectro cada pocos segundos. Con polling, el frontend tendría que consultar al backend repetidamente para detectar nuevos datos, generando tráfico innecesario y latencia.
- WebSocket permite que el backend notifique proactivamente al frontend solo cuando hay datos nuevos, en un modelo push eficiente.

- La librería ws (WebSocket nativo) fue seleccionada sobre Socket.IO por su menor sobrecarga de protocolo: no requiere el handshake adicional de Socket.IO ni sus mecanismos de fallback a long-polling, que son innecesarios en un entorno de red controlada.

6.5.3. PostgreSQL y TimescaleDB para datos de series temporales

La decisión de adoptar PostgreSQL como base de datos principal —y específicamente TimescaleDB como extensión para tablas de series temporales— fue la de mayor impacto en la arquitectura del backend. El sistema comenzó utilizando SQLite, una base de datos sin servidor que almacena toda la información en un solo archivo.

SQLite ofrecía ventajas iniciales de simplicidad: sin necesidad de instalar ni configurar un servidor de base de datos separado, sin autenticación, con despliegue trivial. Sin embargo, a medida que el volumen de datos creció —particularmente en las tablas de estado de sensores y datos de espectro— se manifestaron limitaciones críticas:

- El límite práctico de aproximadamente 2 GB por archivo de base de datos comenzó a causar degradación y bloqueos.
- SQLite no soporta concurrencia real de escritura: múltiples sensores enviando datos simultáneamente generaban contención de bloqueo.
- Las consultas analíticas sobre grandes volúmenes de datos históricos (necesarias para reportes) tenían rendimiento deficiente.

La migración a PostgreSQL resolvió estas limitaciones y habilitó capacidades nuevas: tipos de datos nativos para valores geospaciales y series temporales, transacciones ACID completas en operaciones multi-tabla, índices compuestos para consultas por sensor y rango temporal, y compresión y retención automática de datos antiguos mediante TimescaleDB. La migración se realizó de forma progresiva, actualizando primero la capa de conexión, luego los modelos de datos y finalmente las rutas con consultas directas.

6.5.4. Microservicio de postprocesamiento en Python

El análisis espectral avanzado —detección de emisiones, estimación de piso de ruido por escalones, separación de regiones por valles, cruce con base de datos de licencias— se implementó como un servicio independiente en Python (Flask), separado del backend Node.js.

Esta decisión responde a la naturaleza del procesamiento requerido: el análisis involucra cálculos numéricos intensivos (FFT, integración de potencia, suavizado Savitzky-Golay,

detección de picos, evaluación de cumplimiento) para los cuales el ecosistema Python ofrece librerías maduras y optimizadas (NumPy, SciPy, Pandas). Implementar estas mismas rutinas en Node.js habría requerido reescribir algoritmos complejos o depender de puentes a librerías nativas con mantenimiento incierto.

La separación como microservicio también permite escalar el postprocesamiento de forma independiente, asignar más recursos computacionales al servicio Python sin afectar la capacidad de respuesta del backend, y mantener ciclos de desarrollo separados para la lógica de análisis espectral.

6.5.5. Separación de canales WebSocket para datos y audio

El audio demodulado se transmite por un canal WebSocket independiente del canal de datos de sensores. Esta separación permite tratar cada flujo según sus requisitos específicos:

- El canal de datos transmite eventos discretos (estado, GPS, espectro) con frecuencia de segundos y payloads de tamaño moderado.
- El canal de audio transmite tramas continuas en formato Opus con frecuencia de milisegundos, requiriendo menor latencia y mayor throughput.
- La separación evita que la carga de streaming de audio afecte la entrega de datos de espectro durante un monitoreo, y viceversa.

6.6. Problemas presentados y ajustes realizados

A continuación se documentan los problemas identificados durante el desarrollo y operación de la plataforma, las evidencias recolectadas, el análisis de causas, las decisiones de ajuste tomadas y los resultados obtenidos.

Tabla 6.1: Problemas presentados, decisiones y ajustes realizados en la plataforma web.

Problema	Evidencia	Causa probable	Decisión o Resultado
Límite de capacidad de SQLite causaba bloqueos en producción al alcanzar 2 GB de datos acumulados	Logs de error del servidor, mediciones de tamaño de archivo de base de datos	SQLite no está diseñado para volúmenes de series temporales con múltiples escrituras concurrentes desde sensores	Migración completa a PostgreSQL con extensión TimescaleDB. Se reescribió la capa de conexión, modelos de datos y rutas con consultas directas manteniendo compatibilidad de formatos
Endpoint de campañas para sensores con nombre incorrecto y resultados duplicados en asignaciones múltiples	Logs de consulta de sensores físicos, pruebas de integración	Falta de deduplicación en consulta SQL de asignación sensor-campaña y nomenclatura inconsistente del endpoint	Corrección del nombre del endpoint, incorporación de lógica de deduplicación, adición de filtro opcional por estado y validación de campos requeridos
Timeouts en generación de reportes de cumplimiento para campañas con grandes volúmenes de datos	Errores HTTP 504 en frontend, logs de timeout en Nginx	Timeouts por defecto (30–60 s) insuficientes para el pipeline completo de consulta, post-procesamiento y armado de respuesta	Configuración de timeouts extendidos a 3600 s en servidor Node.js y proxy Nginx (<code>proxy_read_timeout</code> , <code>proxy_connect_timeout</code> , <code>proxy_send_timeout</code>)

Problema	Evidencia	Causa probable	Decisión o Resultado
Sensores que dejaban de reportar permanecían con estado activo en la plataforma	Reportes de personas usuarias, inspección de estados inconsistentes en base de datos	Ausencia de mecanismo automático de verificación de frescura de datos de sensores	Implementación de tarea programada cada 30 s que marca como offline sensores sin reporte reciente y notifica cambios al frontend por WebSocket
Carga inicial lenta del frontend por inclusión de Plotly.js en el bundle principal	Métricas de Lighthouse, reportes de personas usuarias sobre tiempo de carga	Plotly.js (3 MB) se incluía en el bundle principal, bloqueando renderización	Separación de Plotly.js en chunk independiente mediante code splitting en Vite; pantallas sin gráficos no esperan su carga
Errores de Mixed Content en producción con HTTPS: conexiones WebSocket y API bloqueadas por navegador	Errores en consola del navegador, fallos en conexión WebSocket, datos en tiempo real sin actualizar	URLs del backend configuradas como absolutas (HTTP) desde página servida por HTTPS	Migración a URLs relativas (/api, /ws) en variables de entorno de compilación, delegando resolución al proxy Nginx
Inconsistencia de sesión entre múltiples pestañas del navegador	Reportes de personas usuarias, pruebas con múltiples pestañas abiertas	Estado de autenticación y WebSocket gestionado por pestaña sin coordinación	Revisión del flujo de MSAL para propagar eventos de login/logout entre pestañas y manejo de renovación de tokens

6.7. Validación, pruebas o evidencias

La plataforma web ha sido validada mediante múltiples estrategias de verificación a lo largo de su desarrollo y operación:

Pruebas funcionales manuales: cada pantalla del frontend y cada endpoint de la API han sido verificados manualmente durante el desarrollo, cubriendo los flujos principales de autenticación, monitoreo en vivo, creación de campañas, revisión de alertas y generación de reportes.

Pruebas de integración con sensores físicos: se ha verificado la comunicación extremo a extremo con sensores HackRF reales, incluyendo el envío de configuraciones, la recepción de datos de espectro y estado, y la transmisión de audio demodulado AM/FM. El simulador de sensores AM/FM documentado en el backend permite reproducir escenarios de prueba sin hardware físico.

Documentación interactiva de API: la especificación OpenAPI publicada a través de Swagger UI permite verificar la estructura de cada endpoint, los esquemas de solicitud y respuesta, y ejecutar pruebas directamente desde el navegador mediante la función “Try it out”.

Validación en producción: la plataforma se encuentra en operación en el servidor institucional de la ANE, accesible a través de la red interna. El uso continuo por parte del personal técnico constituye una validación operativa de las funcionalidades implementadas.

Logs de servidor: el backend registra cada solicitud HTTP con método, ruta, marca de tiempo y código de respuesta, permitiendo auditoría de la actividad y diagnóstico de problemas. Los logs de Nginx complementan esta trazabilidad con información de tiempos de respuesta y estado de las conexiones.

Mantenimiento automático verificado: la tarea programada de validación de estado de sensores se monitorea a través de los eventos WebSocket que notifican cambios de estado al frontend. Los sensores que dejan de reportar son marcados como offline de forma automática, y este comportamiento ha sido confirmado en operación.

6.8. Aprendizajes y transferencia de conocimiento

Los siguientes aprendizajes se derivan de la experiencia de diseño, implementación y operación de la plataforma web:

1. **Dimensionar la persistencia desde el inicio según la carga esperada:** SQLite fue adecuado para desarrollo y prototipado, pero mantener una base de datos sin servidor para una carga de producción con múltiples escritores concurrentes y grandes volúmenes de series temporales generó un cuello de botella que requirió una

migración significativa. La enseñanza es que la elección de la base de datos debe basarse en el perfil de carga previsto —volumen, concurrencia, tipo de consultas— y no solo en la conveniencia de desarrollo.

2. **La comunicación en tiempo real debe ser push, no pull:** el uso de WebSocket para notificar al frontend sobre nuevos datos y cambios de estado evitó la necesidad de implementar lógica de polling, reduciendo la carga en el servidor y la latencia de visualización. Para sistemas donde los datos fluyen de forma continua desde múltiples fuentes, el patrón push es la estrategia correcta.
3. **Los timeouts deben calibrarse según la operación más costosa:** los valores por defecto de los servidores HTTP (30 segundos) no contemplan operaciones de análisis que involucran consultas complejas, procesamiento numérico y cruce con bases de datos externas. Identificar la operación de mayor duración —los reportes de cumplimiento— y configurar todos los timeouts de la cadena (servidor, proxy) de forma consistente evita fallos intermitentes difíciles de diagnosticar.
4. **La separación de canales por tipo de dato simplifica el manejo de capacidades de servicio:** mantener datos de espectro y audio en canales WebSocket independientes permitió tratar cada flujo con sus propios requisitos de latencia y frecuencia, sin que la carga de streaming de audio afectara la entrega de datos de espectro.
5. **Las URLs relativas eliminan problemas de configuración por entorno:** migrar de URLs absolutas a relativas para API y WebSocket resolvió permanentemente los errores de Mixed Content y permitió que una misma compilación funcione en desarrollo, VPN y producción sin reconfiguración.
6. **La división de código es necesaria en aplicaciones con dependencias de visualización pesadas:** las librerías de gráficos científicos pueden dominar el tamaño del bundle. Planificar code splitting desde el inicio evita resolver problemas de rendimiento cuando la aplicación ya está en producción.
7. **El mantenimiento automático del estado elimina falsos positivos:** sin verificación periódica de la frescura de datos de sensores, los equipos desconectados aparecían como operativos. La automatización de esta verificación, con notificación por WebSocket, eliminó este problema y redujo la carga de monitoreo manual del personal.
8. **La documentación interactiva de API acelera la integración:** contar con Swagger UI permitió que el equipo de frontend y los desarrolladores de los sensores pudieran explorar y probar la API sin depender de documentación estática, reduciendo el tiempo de integración.

6.9. Limitaciones

La plataforma web en su estado actual presenta las siguientes limitaciones:

1. **Base de datos sin alta disponibilidad:** PostgreSQL opera en instancia única, sin replicación ni mecanismo de failover automático. Una falla del servidor de base de datos interrumpiría la totalidad de la plataforma.
2. **Escalamiento vertical del backend:** el backend está diseñado como un solo proceso Node.js. Si bien el pool de 20 conexiones soporta la concurrencia actual, la arquitectura no contempla escalamiento horizontal a múltiples instancias, lo que requeriría un backend compartido para WebSocket (como Redis) y balanceo de carga.
3. **Sin balanceo de carga:** el punto único de entrada a través de Nginx no está configurado para distribuir tráfico entre múltiples réplicas del backend.
4. **Dependencia de conectividad de sensores:** el modelo de operación asume que los sensores consultan periódicamente su configuración. Si un sensor pierde conectividad en el momento de consulta, puede no recibir una campaña programada hasta el siguiente ciclo.
5. **Postprocesamiento síncrono sin cola:** la generación de reportes invoca al servicio Python de forma síncrona. Si el servicio no está disponible, el reporte falla sin mecanismo de reintento automático o encolamiento.
6. **Tablas de series temporales sin optimización completa:** solo `sensor_status` está configurada como hypertable de TimescaleDB. Las tablas `sensor_data` y `sensor_gps`, que también almacenan series temporales, pueden presentar degradación al crecer en volumen.
7. **Tamaño de dependencias de visualización:** Plotly.js (3 MB) sigue siendo una dependencia significativa que afecta la primera visita a las pantallas de monitoreo, incluso estando en un chunk separado.
8. **Sin pruebas automatizadas:** el desarrollo no incluye pruebas unitarias ni de integración automatizadas, dependiendo de validación manual para verificar cambios.
9. **Sin modo offline:** la aplicación requiere conectividad constante con el backend. No hay almacenamiento local de datos para consulta sin conexión.
10. **Sin diseño responsivo para dispositivos móviles:** la interfaz está optimizada para estaciones de escritorio, que es el contexto de uso principal. No se ha priorizado la adaptación a pantallas pequeñas.

6.10. Recomendaciones específicas

Con base en las limitaciones identificadas y la experiencia acumulada, se proponen las siguientes recomendaciones para la evolución de la plataforma:

1. **Alta disponibilidad de base de datos:** configurar replicación streaming en PostgreSQL con al menos una réplica en caliente, permitiendo continuidad operativa ante falla del servidor principal.
2. **Escalamiento horizontal del backend:** evaluar la introducción de Redis como backend compartido para WebSocket y como caché de datos frecuentes, habilitando múltiples instancias del servidor Node.js coordinadas detrás de un balanceador de carga.
3. **Optimización de todas las tablas de series temporales:** migrar `sensor_data` y `sensor_gps` a hypertables de TimescaleDB, y configurar políticas de compresión y retención automática para todas las tablas de series temporales.
4. **Resiliencia del postprocesamiento:** implementar una cola de trabajos (por ejemplo, Bull con Redis) para las solicitudes de reportes, con reintentos automáticos con backoff exponencial cuando el servicio Python no esté disponible.
5. **Pruebas automatizadas:** introducir pruebas unitarias con Vitest/Jest para el frontend y backend, y pruebas de integración para los flujos críticos (autenticación, monitoreo en vivo, creación de campañas, generación de reportes).
6. **Evaluación de alternativas de visualización:** analizar la viabilidad de reemplazar Plotly.js por ECharts o una implementación basada en Canvas/WebGL para reducir el tamaño del bundle sin perder capacidades de visualización.
7. **Monitoreo operacional:** instrumentar el backend con métricas de Node.js (consumo de memoria, event loop lag, latencia de respuesta, conexiones activas) y exportarlas a un sistema de monitoreo institucional.
8. **Internacionalización preparatoria:** si bien el contexto actual es exclusivamente colombiano, estructurar los textos de la interfaz de forma que una futura traducción no requiera reescribir componentes, utilizando una capa de localización (i18n).
9. **Revisión de seguridad:** auditar los endpoints de administración para verificar que todos requieran autenticación y autorización por rol, y documentar explícitamente el modelo de seguridad (endpoints públicos de ingesta, endpoints autenticados de administración).
10. **Documentación operativa:** elaborar guías de operación para el personal técnico de la ANE, cubriendo procedimientos de despliegue, respaldo, recuperación y diagnóstico de problemas comunes.

6.11. Repositorios

El software de la plataforma web está organizado en dos módulos dentro del repositorio del sistema. A continuación se presentan las fichas técnicas de cada componente.

Tabla 6.2: Repositorio asociado al frontend de la plataforma web.

Nombre del software	ANE Frontend
Repositorio	Repositorio pendiente de publicación
Rama, versión o commit	main
Responsable	Equipo de desarrollo de plataforma
Función dentro del sistema	Interfaz web para la operación y administración del sistema de monitoreo espectral
Entradas	Interacciones de la persona usuaria, datos en tiempo real por WebSocket, respuestas de API REST
Salidas	Visualizaciones de espectro y mapas, formularios de configuración, reportes de cumplimiento, streaming de audio
Dependencias	React 19, TypeScript, Vite 5, Tailwind CSS 3, Plotly.js, Leaflet, MSAL, Axios, Nginx
Estado actual	En producción en servidor institucional privado

Tabla 6.3: Repositorio asociado al backend de la plataforma web.

Nombre del software	ANE Backend
Repositorio	Repositorio pendiente de publicación
Rama, versión o commit	main
Responsable	Equipo de desarrollo de plataforma
Función dentro del sistema	Centro de coordinación: API REST, WebSocket, persistencia, gestión de campañas, alertas y reportes
Entradas	Solicitudes HTTP del frontend y sensores; datos de espectro, GPS, estado y audio desde sensores RF
Salidas	Respuestas JSON, eventos WebSocket, reportes de cumplimiento normativo
Dependencias	Node.js 18+, Express, TypeScript, PostgreSQL 13+, TimescaleDB (opcional), ws, Swagger/OpenAPI, Azure AD, servicio Python de postprocesamiento
Estado actual	En producción en servidor institucional privado

Capítulo 7

Análisis espectral y localización en la nube

7.1. Propósito del capítulo

Esta sección documenta el módulo de postprocesamiento espectral desarrollado para analizar, en un entorno centralizado, las capturas generadas por el sistema de monitoreo. El objetivo de este avance es explicar cómo el software recibe una traza espectral, organiza la información de frecuencia y amplitud, selecciona un modo de operación y obtiene los primeros parámetros técnicos de las emisiones detectadas.

En esta etapa se describe el proceso hasta la estimación de la **frecuencia central**, el **ancho de banda**, la **potencia** y, cuando la captura corresponde a un caso aplicable, **MER/BER**. La verificación detallada de cumplimiento contra licencias, el uso formal de DANE/municipio, la localización administrativa, las pruebas finales y las recomendaciones quedan como continuación de esta misma sección.

El propósito del módulo no es reemplazar la adquisición SDR realizada en el borde, sino convertir las capturas recibidas en resultados interpretables para la plataforma. Por esta razón, el capítulo se enfoca en la función del software dentro del flujo general del sistema, en sus entradas y salidas principales, y en las decisiones iniciales usadas para diferenciar ruido, picos solicitados y emisiones candidatas.

7.2. Contexto dentro del sistema general

El sistema completo puede entenderse como una cadena de sensado, transmisión, procesamiento y visualización. En el borde, el nodo de monitoreo recibe señales de radiofrecuencia mediante el hardware SDR y genera una captura espectral. Posteriormente, esa captura

se envía hacia la plataforma central o se analiza de forma local mediante archivos de entrada. El módulo descrito en esta sección se ubica después de la adquisición y antes de la presentación de resultados al usuario.

La información que llega al módulo corresponde principalmente a una serie de muestras espectrales asociadas a un rango de frecuencias. Con esos datos, el software reconstruye el eje de frecuencia, identifica regiones de interés y entrega resultados estructurados. Estos resultados pueden ser usados por una interfaz web, por un servicio de consulta o por un proceso posterior de validación.

En este punto del desarrollo, la palabra localización se interpreta de forma limitada. El módulo no calcula una posición geográfica exacta de la emisión ni realiza triangulación con varios nodos. La relación espacial disponible se maneja a través de filtros administrativos, como código DANE, lista de DANEs o municipio, los cuales permiten contextualizar una medición dentro de una zona de análisis. El desarrollo detallado de esta parte queda para la continuación de la sección.

7.3. Arquitectura del módulo de postprocesamiento

El desarrollo se organiza como un módulo de Python separado en archivos con responsabilidades específicas. Esta división facilita el mantenimiento, la prueba por componentes y la integración posterior con una plataforma web o con una API.

Tabla 7.1: Archivos principales del módulo de postprocesamiento espectral.

Archivo	Función dentro del módulo
<code>main.py</code>	Permite ejecutar el procesamiento desde consola. Lee un archivo JSON o una trama en línea, interpreta argumentos como picos, cumplimiento, DANE, umbral y rutas auxiliares, y entrega una respuesta estructurada.
<code>server_flask.py</code>	Expone el procesamiento mediante una API basada en Flask. Permite recibir solicitudes HTTP con una captura espectral y parámetros de análisis.
<code>src/payload_parser.py</code>	Convierte el contenido del JSON de entrada en un objeto espectral interno. Valida que existan las muestras y los límites de frecuencia necesarios.
<code>src/spectrum_frame.py</code>	Define la estructura <code>SpectrumFrame</code> , que contiene amplitudes, frecuencia inicial, frecuencia final, eje de frecuencias y resolución espectral.
<code>src/processor.py</code>	Orquesta el flujo principal. Normaliza la entrada, selecciona el modo de operación, aplica corrección si existe, llama a los algoritmos de detección y arma la salida final.
<code>src/spectral_analysis.py</code>	Contiene las funciones de análisis espectral, detección de picos, segmentación de emisiones y estimación de parámetros como frecuencia central, ancho de banda, potencia, MER y BER cuando aplica.
<code>src/calibration_io.py</code>	Contiene utilidades para trabajar con archivos auxiliares, como bases de licencias o información de comparación. En este avance solo se menciona su papel dentro de la arquitectura.
<code>src/power_utils.py</code>	Contiene funciones asociadas al cálculo robusto de potencia. Integra la energía de la emisión en dominio lineal y evita reportar valores no finitos cuando la captura no permite una estimación confiable.

La arquitectura permite ejecutar el mismo procesamiento desde dos frentes: por consola, usando `main.py`, o como servicio, usando `server_flask.py`. En ambos casos, la función central es `process_input`, definida en `src/processor.py`, porque allí se decide el modo de análisis y se construye la respuesta final.

7.4. Flujo general de procesamiento

El flujo de trabajo inicia con una captura espectral en formato JSON. El sistema espera una lista de valores espectrales y dos frecuencias límite que definan el intervalo analizado. A partir de esa información, se construye una representación interna de la traza y se ejecuta el modo de operación correspondiente.

De forma resumida, el flujo es el siguiente:

1. Recepción del JSON espectral o de una trama equivalente.
2. Lectura de los campos principales de frecuencia y amplitud.
3. Construcción del objeto `SpectrumFrame`.
4. Aplicación opcional de corrección de ganancia, si se entrega un archivo de corrección.
5. Selección del modo de operación según los picos recibidos y el valor de cumplimiento.
6. Cálculo del umbral de detección o uso del umbral configurado por el usuario.
7. Búsqueda de picos o regiones candidatas de emisión.
8. Estimación de la frecuencia central de cada emisión candidata.
9. Estimación del ancho de banda mediante ancho a $-x$ dB u OBW cuando está disponible.
10. Estimación de potencia integrada de la emisión en la región medida.
11. Estimación opcional de MER/BER cuando la señal analizada corresponde a un caso TDT compatible.
12. Generación de una salida estructurada con el modo usado y los resultados medidos.

Este flujo conserva una separación clara entre los datos de entrada, la representación interna y los resultados. Esa decisión es importante porque evita que cada parte del sistema tenga que interpretar directamente el JSON original.

7.5. Entradas del sistema

Las entradas principales del módulo son la captura espectral y algunos parámetros de control. No todos los campos son obligatorios en todos los modos; algunos se usan solo cuando se quiere hacer análisis por picos o preparar una validación posterior de cumplimiento.

Tabla 7.2: Entradas principales consideradas en este avance del módulo.

Entrada	Tipo esperado	Descripción
<code>Pxx</code>	Lista numérica	Vector de amplitudes o potencias espectrales de la captura. Es la señal base que se analiza.
<code>start_freq_hz</code>	Numérico	Frecuencia inicial de la captura, expresada en Hz.
<code>end_freq_hz</code>	Numérico	Frecuencia final de la captura, expresada en Hz.
<code>picos</code>	Lista numérica	Frecuencias específicas solicitadas por el usuario. Si se entregan, el sistema entra al modo de análisis por picos.
<code>cumplimiento</code>	Entero 0/1	Bandera que indica si se desea ejecutar el modo de cumplimiento cuando no se entregan picos.
<code>corr</code>	Ruta de archivo	Archivo opcional de corrección de ganancia. Permite ajustar la traza antes del análisis.
<code>lic</code>	Ruta de archivo	Archivo opcional de licencias. En este avance solo se registra como dependencia del modo de cumplimiento.
<code>dane,</code> <code>municipio</code> <code>umbrales_db</code>	<code>danes,</code> Texto o lista	Filtros administrativos que permiten contextualizar la captura en una zona de análisis.
<code>delta_fc_khz</code>	Numérico opcional	Umbral en dB sobre el piso de ruido usado para decidir si una región puede considerarse candidata de emisión.
<code>delta_bw_khz</code>	Numérico opcional	Tolerancia de frecuencia central, en kHz, usada para comparar o emparejar frecuencias cercanas.
		Tolerancia de ancho de banda, en kHz. En este avance se menciona como parámetro disponible, pero no se desarrolla la evaluación normativa completa.

El parser de entrada también acepta algunos alias de nombres para los campos principales, como `pxx`, `PSD`, `f_start_hz` o `f_stop_hz`. Esta flexibilidad permite integrar capturas provenientes de fuentes distintas sin cambiar la estructura general del procesamiento.

7.6. Salidas del sistema

La salida del módulo se entrega como una estructura de datos con información general del procesamiento y una lista de resultados. En este avance se documentan las salidas relacionadas con modo, picos, estado de detección, frecuencia central, ancho de banda, potencia y MER/BER cuando el caso aplica.

Tabla 7.3: Salidas generales del procesamiento.

Salida	Tipo esperado	Descripción
mode	Texto	Modo de operación seleccionado: <code>all_emissions</code> , <code>peaks</code> o <code>compliance</code> .
cumplimiento	Entero 0/1	Valor de cumplimiento recibido en la entrada.
picos_count	Entero	Cantidad de picos entregados por el usuario.
picos	Lista numérica	Picos solicitados originalmente, cuando existen.
umbral	Numérico	Umbral absoluto usado para apoyar la detección.
num_emissions	Entero	Cantidad de emisiones o candidatos encontrados.
results	Lista de objetos	Resultados individuales de cada emisión, pico solicitado o candidato analizado.

Tabla 7.4: Campos principales dentro de cada resultado.

Campo	Tipo esperado	Descripción
status	Texto	Indica si se encontró una emisión o si el punto analizado se consideró ruido.
reason	Texto	Explicación breve cuando no se detecta una emisión válida.
requested_pico	Numérico	Pico solicitado por el usuario en modo de análisis por picos.
matched_peak_hz	Numérico	Pico detectado más cercano al pico solicitado.

Campo	Tipo esperado	Descripción
<code>delta_match_hz</code>	Numérico	Diferencia entre el pico solicitado y el pico detectado asociado.
<code>fc_hz</code>	Numérico	Frecuencia central estimada de la emisión, expresada en Hz.
<code>fc_mhz</code>	Numérico	Frecuencia central estimada de la emisión, expresada en MHz.
<code>bw_hz</code>	Numérico	Ancho de banda reportado para la emisión, expresado en Hz.
<code>bw_khz</code>	Numérico	Ancho de banda reportado para la emisión, expresado en kHz.
<code>power_dbm</code>	Numérico o nulo	Potencia estimada de la emisión. Si el cálculo no es finito, no se fuerza un valor artificial.
<code>snr_db</code>	Numérico opcional	Relación señal a ruido estimada cuando la función de medición la entrega.
<code>mer_db</code>	Numérico opcional	MER estimado para señales compatibles, especialmente TDT. Solo aparece si el cálculo aplica.
<code>ber_est</code>	Numérico opcional	BER estimado para señales compatibles, especialmente TDT. Solo aparece si el cálculo aplica.

La estructura de salida permite que otro módulo consuma los resultados sin depender de mensajes impresos en consola. Esto es necesario para integrarlo con una plataforma web, una base de datos o una capa de visualización.

7.7. Modos de operación

El módulo selecciona automáticamente el modo de operación usando dos elementos de entrada: la lista de picos y la bandera de cumplimiento. La regla implementada es simple: si hay picos, se prioriza el análisis por picos; si no hay picos y cumplimiento vale uno, se activa el modo de cumplimiento; si no hay picos y cumplimiento vale cero, se detectan todas las emisiones candidatas.

Tabla 7.5: Modos de operación del módulo de postprocesamiento.

Modo	Condición de entrada	Uso principal
<code>all_emissions</code>	No se entregan picos y <code>cumplimiento = 0</code> .	Detectar automáticamente emisiones candidatas dentro de toda la captura espectral y reportar sus primeros parámetros técnicos.
<code>peaks</code>	Se entrega una lista de picos, sin importar el valor de cumplimiento.	Analizar frecuencias específicas solicitadas por el usuario y determinar si cerca de ellas existe una emisión o si el punto se comporta como ruido.
<code>compliance</code>	No se entregan picos y <code>cumplimiento = 1</code> .	Preparar el procesamiento para comparar emisiones medidas contra información de referencia. En este avance se llega hasta la medición técnica de la emisión y la estimación MER/BER si aplica, sin desarrollar todavía el análisis completo de licencias.

7.7.1. Detección automática de emisiones

En el modo `all_emissions`, el sistema analiza toda la traza espectral. Primero estima un umbral de detección a partir del piso de ruido o del valor indicado por el usuario mediante `umbral_db`. Después busca picos o regiones candidatas y, para cada una, mide frecuencia central, ancho de banda y potencia. Si la emisión corresponde a un caso TDT compatible, también puede estimar MER y BER.

Este modo es útil cuando el usuario no sabe previamente en qué frecuencias existen emisiones de interés. La salida principal de esta etapa es una lista de candidatos con su estado y sus parámetros espectrales iniciales.

7.7.2. Análisis por picos solicitados

En el modo `peaks`, el usuario entrega una o varias frecuencias de interés. El sistema ubica la frecuencia solicitada dentro del eje de la captura y busca un pico cercano que pueda representar una emisión real. Si encuentra una región suficientemente consistente, reporta los parámetros medidos de la emisión. Si no encuentra evidencia suficiente, el resultado se marca como ruido o como pico no asociado a una emisión clara.

Este modo es conveniente cuando se desea comprobar una frecuencia puntual, por ejemplo

una frecuencia observada visualmente en una gráfica o una frecuencia que se sospecha activa.

7.7.3. Modo de cumplimiento

En el modo **compliance**, el sistema queda preparado para relacionar las emisiones detectadas con una base de referencia, normalmente una base de licencias. Para entrar en este modo no deben entregarse picos y la bandera **cumplimiento** debe estar activa.

En este avance no se desarrolla la comparación completa de cumplimiento. Solo se documenta que el modo existe dentro del enrutamiento del software y que utiliza la misma base inicial de detección espectral para obtener los parámetros medidos que luego pueden ser comparados con valores nominales.

7.8. Análisis espectral realizado: frecuencia central, ancho de banda, potencia y MER/BER

El análisis espectral del módulo se centra en convertir una región detectada de la traza en parámetros técnicos que puedan ser interpretados por la plataforma. En este avance se documentan cuatro salidas principales: frecuencia central, ancho de banda, potencia y, solo cuando la captura lo permite, MER/BER. Estos valores no se escriben de forma manual, sino que se derivan de la región espectral detectada por el algoritmo.

7.8.1. Frecuencia central

La frecuencia central representa la frecuencia principal o representativa de una emisión detectada dentro de la captura. En el módulo, esta frecuencia se entrega principalmente en dos unidades: Hz mediante **fc_hz** y MHz mediante **fc_mhz**.

El proceso inicia con el eje de frecuencias generado por **SpectrumFrame**. Si la captura contiene N muestras, el objeto construye un vector de frecuencias entre **start_freq_hz** y **end_freq_hz**. Con ese eje, cada muestra de amplitud queda asociada a una frecuencia específica.

Cuando el sistema detecta un pico candidato, se toma una frecuencia inicial de referencia desde el eje de frecuencias. Luego se delimita una región de análisis alrededor del candidato y se llama a las funciones de medición espectral. El resultado final de esa medición se almacena como frecuencia central de la emisión.

En términos operativos, la frecuencia central puede provenir de tres situaciones:

- En detección automática, se obtiene a partir del pico o segmento candidato encontrado por el algoritmo.
- En análisis por picos, se calcula a partir de la emisión real encontrada cerca de la frecuencia solicitada.
- En modo de cumplimiento, se conserva como frecuencia central medida para que posteriormente pueda compararse con una frecuencia nominal.

La frecuencia central no debe interpretarse simplemente como el punto más alto de la traza. En algunas capturas, especialmente cuando hay señales anchas o regiones con ruido, el punto de mayor amplitud puede no representar de forma adecuada el centro de la emisión. Por eso el módulo no solo toma un valor visual, sino que usa la región detectada para estimar una frecuencia más representativa.

Tabla 7.6: Campos asociados a la frecuencia central.

Campo	Unidad	Interpretación
<code>fc_hz</code>	Hz	Frecuencia central estimada en la unidad base usada internamente por el procesamiento.
<code>fc_mhz</code>	MHz	Versión escalada de <code>fc_hz</code> , más cómoda para reportes y lectura humana.
<code>matched_peak_hz</code>	Hz	Frecuencia de pico asociada a una solicitud del usuario en modo peaks . Puede servir como referencia inicial, pero no siempre reemplaza la frecuencia central estimada.
<code>delta_match_hz</code>	Hz	Diferencia entre la frecuencia solicitada y el pico asociado. Ayuda a evaluar si el emparejamiento fue cercano.

7.8.2. Ancho de banda

El ancho de banda describe la extensión espectral ocupada por una emisión. En el software, este valor se obtiene después de delimitar la región asociada al pico o segmento candidato. La función de medición calcula dos referencias: un ancho a $-x$ dB respecto al pico y un ancho de banda ocupado u OBW, que representa el intervalo que contiene un porcentaje definido de la potencia de la emisión.

Para el reporte final, el procesador selecciona el ancho de banda más conveniente mediante una regla conservadora: si existe OBW válido, se prefiere este valor porque suele ser más estable en señales con mesetas, varios picos o modulaciones anchas. Si el OBW no está

disponible, el sistema usa el ancho a $-x$ dB. Esta decisión evita depender de un solo bin o de una lectura visual de la gráfica.

El resultado se reporta como `bw_hz` y `bw_khz`. El primer campo mantiene la unidad interna del procesamiento y el segundo facilita la lectura del reporte. En modo de cumplimiento, el mismo valor medido se conserva como `bw_medido_kHz`, pero la comparación normativa completa queda para una etapa posterior de la sección.

Tabla 7.7: Campos asociados al ancho de banda.

Campo	Unidad	Interpretación
<code>bandwidth_xdb_hz</code>	Hz	Ancho calculado a $-x$ dB respecto al pico de la emisión.
<code>obw_hz</code>	Hz	Ancho de banda ocupado, calculado a partir de la potencia acumulada de la región.
<code>bw_hz</code>	Hz	Ancho de banda seleccionado para reportar la emisión.
<code>bw_khz</code>	kHz	Versión del ancho de banda reportado en una unidad más cómoda para lectura humana.
<code>bw_medido_kHz</code>	kHz	Nombre usado en modo de cumplimiento para conservar el ancho de banda medido.

7.8.3. Potencia de la emisión

La potencia se estima a partir de la región espectral asociada a la emisión. Para evitar errores de interpretación, el cálculo no se realiza directamente en dBm como si fuera una suma lineal. El procedimiento convierte las muestras al dominio lineal, integra o suma la contribución de los bins dentro de la región seleccionada y finalmente devuelve el resultado en dBm.

El módulo usa funciones de apoyo para hacer el cálculo más robusto. Por ejemplo, la función de potencia robusta permite integrar usando el ancho de bin del `SpectrumFrame` o una estimación basada en el eje de frecuencias. Si la captura no permite calcular una potencia finita, el procesador evita reportar `-inf`, `NaN` o valores artificiales, y conserva el resultado como nulo. Esto es importante porque una potencia no finita no debe interpretarse como una medición real.

En los modos `all_emissions` y `peaks`, la salida principal se reporta como `power_dbm`. En modo de cumplimiento, la misma magnitud se puede conservar como `p_medida_dbm`. En este avance se documenta la medición de potencia, pero no se desarrolla todavía la evaluación frente a potencia nominal de una licencia.

Tabla 7.8: Campos asociados a potencia.

Campo	Unidad	Interpretación
<code>power_dbm</code>	dBm	Potencia estimada de la emisión en los modos de detección general y análisis por picos.
<code>p_medida_dBm</code>	dBm	Nombre usado en modo de cumplimiento para conservar la potencia medida.
<code>channel_power_dbm</code>	dBm	Potencia de canal calculada durante la medición de parámetros de la emisión.
<code>snr_db</code>	dB	Relación señal a ruido estimada cuando la función de análisis la entrega.

7.8.4. MER y BER cuando aplica

El módulo también incluye una estimación opcional de MER y BER para señales compatibles con el caso TDT. Este cálculo no se aplica a todas las emisiones detectadas. La función primero verifica si la frecuencia central de la región cae dentro del rango TDT definido en el código y si el ancho de la banda es plausible para ese tipo de señal. Si esas condiciones no se cumplen, la función retorna un resultado vacío y el reporte no incluye campos de MER/BER.

Cuando el caso sí aplica, el sistema toma la banda medida como región de señal y usa regiones laterales o información fuera de banda como referencia de ruido. Con esa relación se estima una magnitud de calidad equivalente a MER en dB y luego se calcula un BER estimado para una modulación M-QAM, usando $M = 64$ en el flujo implementado para TDT.

Estos valores deben interpretarse como estimaciones de calidad obtenidas desde la traza espectral disponible, no como una demodulación completa de la señal. Por lo tanto, son útiles para orientar el diagnóstico de una captura TDT, pero deben validarse con mediciones especializadas si se requiere una certificación formal.

Tabla 7.9: Campos asociados a MER/BER cuando la estimación aplica.

Campo	Unidad	Interpretación
mer_db	dB	MER estimado de la señal compatible. Se reporta solo si la función encuentra condiciones válidas para el cálculo.
ber_est	Adimensional	BER estimado a partir de la relación de calidad calculada para la señal.
f_center_hz	Hz	Frecuencia central de la banda usada internamente para el cálculo de MER/BER.
band_start_hz	Hz	Límite inferior de la banda evaluada.
band_stop_hz	Hz	Límite superior de la banda evaluada.

Capítulo 8

Protocolos de prueba en nube y borde

Esta sección debe describir cómo se valida el sistema completo y sus componentes individuales, definiendo procedimientos de prueba, métricas, criterios de aceptación y condiciones bajo las cuales se ejecutan los experimentos.

Como placeholder, el texto actual sirve para organizar la documentación de pruebas funcionales, de rendimiento y de integración tanto en el entorno de borde como en la infraestructura desplegada en la nube.

Puntos sugeridos para completar

- Objetivos de prueba y alcance de validación.
- Escenarios experimentales en borde y en nube.
- Métricas de desempeño, confiabilidad y latencia.
- Procedimiento de ejecución y captura de evidencias.
- Resultados esperados, criterios de aceptación y hallazgos.

Capítulo 9

Integración general del sistema

Esta sección debe consolidar la visión de conjunto del sistema una vez integrados los módulos de hardware, adquisición SDR, procesamiento en borde, servicios en nube y análisis posterior.

El placeholder permite dejar una base para explicar la secuencia operativa del sistema, los puntos de acoplamiento entre componentes, los riesgos de integración y las decisiones tomadas para lograr una operación coordinada.

Puntos sugeridos para completar

- Flujo extremo a extremo del sistema.
- Dependencias entre subsistemas y puntos de integración.
- Mecanismos de sincronización, intercambio y supervisión.
- Problemas encontrados durante la integración y su solución.
- Estado actual de madurez e interoperabilidad del sistema.

Capítulo 10

Conclusiones generales

Esta sección debe sintetizar los principales resultados técnicos del proyecto y resumir qué se logró demostrar con el diseño, la implementación, la integración y las pruebas realizadas sobre el sistema.

Como placeholder, se reserva este espacio para formular conclusiones claras, relacionadas con el cumplimiento de objetivos, las capacidades alcanzadas y las limitaciones que todavía deben considerarse en iteraciones futuras.

Puntos sugeridos para completar

- Principales logros técnicos del sistema.
- Relación entre objetivos planteados y resultados obtenidos.
- Fortalezas de la arquitectura o implementación.
- Limitaciones persistentes o aspectos no resueltos.
- Valor del documento como base para etapas posteriores.

Capítulo 11

Recomendaciones

Esta sección debe presentar sugerencias concretas para mejorar el sistema, fortalecer su documentación, ampliar su capacidad operativa o reducir riesgos identificados durante el desarrollo y la validación.

El placeholder actual deja preparada una estructura para priorizar acciones futuras en hardware, software, procesamiento, despliegue y trabajo colaborativo entre los integrantes del proyecto.

Puntos sugeridos para completar

- Mejoras prioritarias de corto plazo.
- Recomendaciones de mediano plazo para escalamiento.
- Ajustes sugeridos en hardware o adquisición de datos.
- Recomendaciones para pruebas, monitoreo y mantenimiento.
- Oportunidades de investigación o extensión futura.

Capítulo 12

Referencias

Esta sección debe reunir las fuentes técnicas, académicas y documentales utilizadas para sustentar decisiones de diseño, selección de tecnologías, métodos de análisis y procedimientos de implementación o prueba.

Como placeholder, este bloque ayuda a reservar la estructura necesaria para incorporar referencias bibliográficas consistentes y trazables, siguiendo el formato de citación que el equipo o la institución definan posteriormente.

Puntos sugeridos para completar

- Artículos, libros o reportes técnicos relevantes.
- Documentación oficial de hardware, SDR y plataformas en nube.
- Referencias sobre algoritmos de análisis espectral y localización.
- Normas, manuales o estándares aplicables.
- Formato de citación adoptado por el documento final.

Apéndice A

Técnica de desarrollo en grupo usando agentes IA

A.1. Propósito del capítulo

Este anexo tiene como objetivo documentar la metodología de trabajo colaborativo y la infraestructura técnica utilizada por el equipo de desarrollo al integrarse con agentes de Inteligencia Artificial (IA). Se busca proporcionar una guía clara sobre la instalación, configuración y mejores prácticas de *prompting* para asegurar que el uso de estas herramientas sea trazable, repetible y de alta calidad dentro del ciclo de vida del proyecto.

A.2. Contexto dentro del sistema general

En el desarrollo de sistemas complejos como el de adquisición SDR y plataformas *cloud*, el uso de agentes IA (como Gemini CLI y Copilot CLI) permite acelerar la automatización de flujos de trabajo, la revisión de código y la generación de documentación técnica. Esta técnica no reemplaza la supervisión humana, sino que actúa como un multiplicador de capacidades, integrando razonamiento multi-agente para la toma de decisiones técnicas y el análisis de contexto.

A.3. Desarrollo técnico: Ecosistema de Agentes CLI

El equipo utiliza una combinación de herramientas de terminal que permiten una interacción directa con modelos de lenguaje avanzados.

A.3.1. Instalación y Configuración

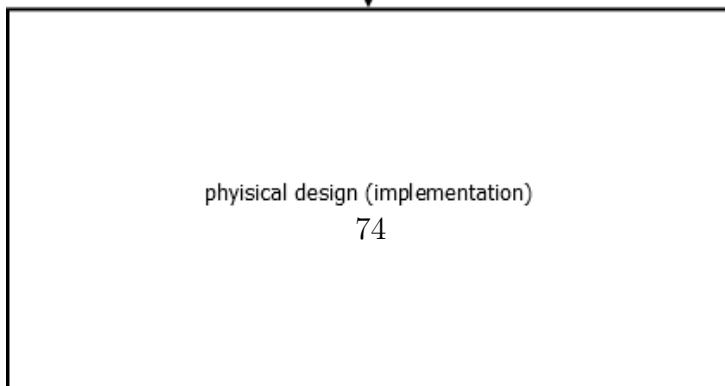
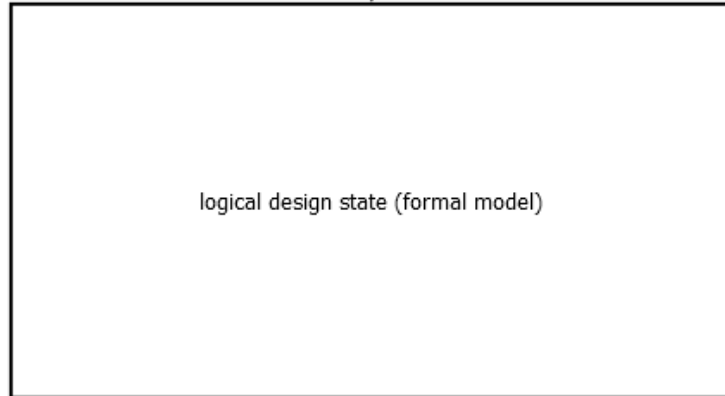
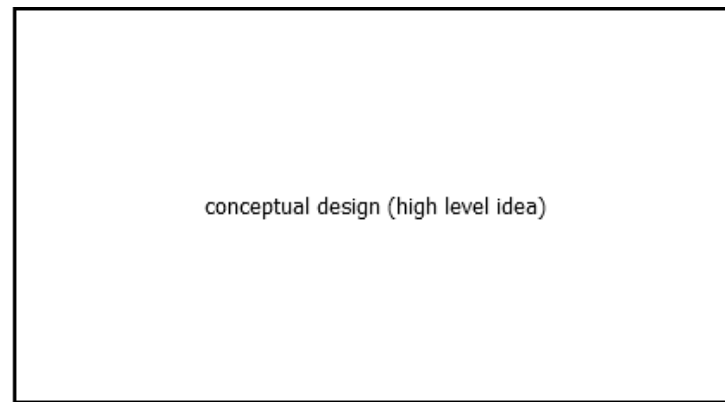
A continuación se detallan los comandos principales para habilitar el entorno de agentes:

- **GitHub Copilot CLI:** Utilizado principalmente para asistencia en codificación y automatización de tareas de repositorio.
 - Instalación: `npm install -g @github/copilot`
 - Autenticación: `copilot /login`
- **Gemini CLI:** Utilizado para tareas multimodales, gestión de habilidades (*skills*) y análisis de arquitectura.
 - Instalación: `npm install -g @google/gemini-cli`
 - Inicio: `gemini`

A.3.2. Arquitectura Multi-Agente

El sistema se diseña bajo un esquema de roles especializados que colaboran para resolver tareas complejas:

- **Agente Coordinador:** Asigna roles y selecciona el modelo más apto.
- **Agente de Interpretación:** Analiza el contexto de la petición.
- **Agente de Backend:** Gestiona el almacenamiento y la comunicación con APIs.
- **Agente de Procesamiento:** Maneja tareas específicas de señales (audio/espectro).



A.4. Decisiones de diseño: Metodología de Prompting

Para minimizar alucinaciones y maximizar la coherencia de las respuestas, se ha adoptado una estructura obligatoria para cada interacción con los agentes, basada en el modelo **ROLE** → **OBJECTIVE** → **CONTEXT** → **REASONING**.

Tabla A.1: Componentes de un Prompt de alta calidad.

Componente	Descripción
Role	Define quién debe responder (e.g., "Senior Software Developer").
Objective	Define qué se quiere lograr de forma clara y concisa.
Context	Proporciona archivos relevantes, restricciones y entorno.
Reasoning	Solicita al agente explicar su lógica antes de entregar el resultado.
Output Format	Especifica el formato deseado (JSON, LaTeX, C code, etc.).

A.5. Problemas presentados y ajustes realizados

Durante la implementación de esta metodología, se identificaron los siguientes desafíos:

Tabla A.2: Problemas en la colaboración con IA y ajustes aplicados.

Problema	Evidencia	Causa probable	Decisión o Resultado
Alucinaciones técnicas	Código que usa librerías inexistentes.	Contexto insuficiente en el prompt inicial.	Implementar el patrón ROLE-CONTEXT . Mayor precisión en el código generado.
Pérdida de hilo conductor	El agente olvidó restricciones previas.	Ventana de contexto saturada.	Uso de comandos /compact y archivos .geminiignore . Sesiones más largas y eficientes.

A.6. Validación y Evidencias

La metodología se valida mediante la revisión humana de cada entrega. El flujo de trabajo colaborativo asegura que ningún cambio se integre sin pasar por un ciclo de validación

cruzada entre agentes y desarrolladores.

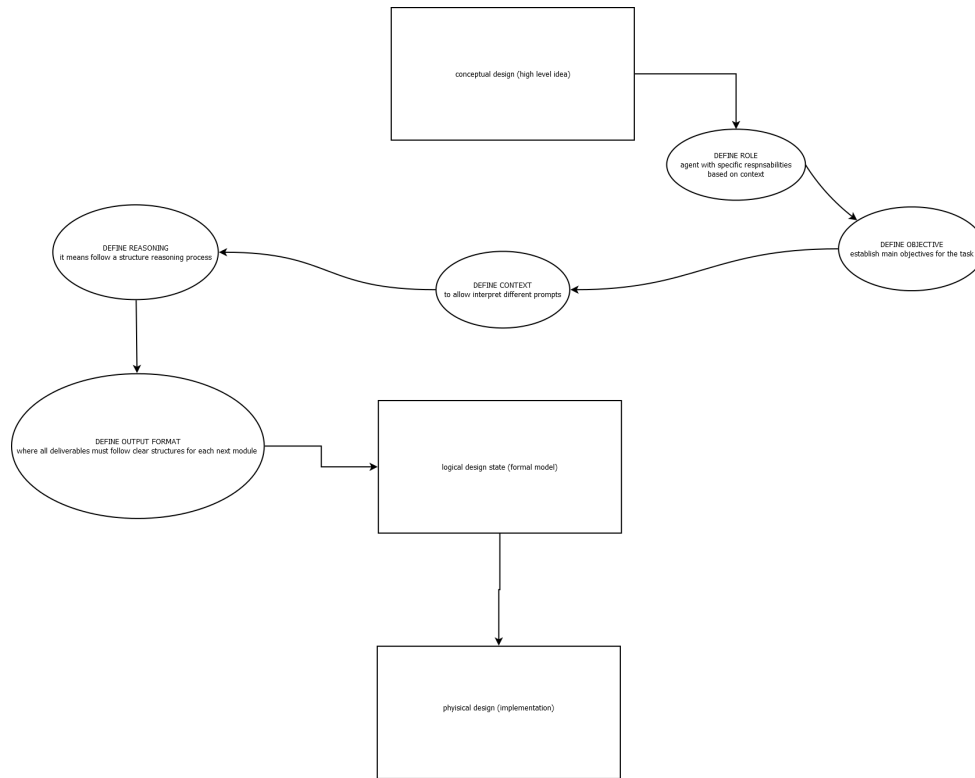


Figura A.2: Roles asignados y estados de diseño en el flujo de trabajo IA. Fuente: Elaboración propia.

A.7. Aprendizajes y transferencia de conocimiento

- **Criterio de Realidad:** La IA es un asistente, no un ejecutor autónomo sin supervisión. La validación humana es mandatoria.
- **Modulariad:** Es preferible pedir tareas pequeñas y específicas que una solución completa y compleja en un solo prompt.
- **Documentación:** Mantener archivos `GEMINI.md` o `AGENTS_CLI.md` en la raíz del proyecto ayuda a los agentes a entender rápidamente las convenciones del equipo.

A.8. Recomendaciones específicas

Se recomienda la rotación de roles entre los desarrolladores para familiarizarse con las capacidades de diferentes *backends* (GPT-4o, Claude 3.5 Sonnet, Gemini 1.5 Pro) según

la naturaleza de la tarea.

Apéndice B

Diagramas adicionales

Este anexo está destinado a reunir diagramas complementarios que apoyen la comprensión del sistema, pero que por extensión o nivel de detalle no convenga ubicar en el cuerpo principal del documento.

Como placeholder, aquí pueden integrarse esquemas de arquitectura, diagramas de bloques, secuencias de operación, topologías de red o representaciones de flujo de datos más detalladas que las incluidas en las secciones centrales.

Puntos sugeridos para completar

- Diagramas de arquitectura extendida.
- Diagramas de despliegue o conectividad.
- Secuencias operativas o flujos de eventos.
- Esquemas detallados de hardware o adquisición.
- Figuras de apoyo para pruebas o integración.

Apéndice C

Fragmentos de código relevantes

Este anexo debe agrupar fragmentos de código que ayuden a ilustrar decisiones de implementación, configuraciones importantes o partes clave del sistema sin sobrecargar el desarrollo principal del informe.

El placeholder actual reserva una estructura útil para incluir ejemplos de scripts, configuraciones, endpoints, pipelines de procesamiento o comandos de despliegue con su respectiva explicación técnica.

Puntos sugeridos para completar

- Código de adquisición o preprocesamiento en borde.
- Fragmentos de servicios, APIs o componentes web.
- Scripts de automatización o despliegue.
- Configuraciones técnicas relevantes para reproducibilidad.
- Comentarios sobre decisiones de implementación.

Apéndice D

Manuales o guías de uso

Este anexo debe compilar instrucciones operativas para instalar, ejecutar, mantener o demostrar el sistema, orientadas a integrantes del equipo, evaluadores o futuros usuarios que necesiten reproducir el entorno de trabajo.

Como placeholder, este archivo deja definido el espacio para documentar procedimientos paso a paso, dependencias, configuraciones previas y recomendaciones de uso seguro del sistema.

Puntos sugeridos para completar

- Requisitos previos de instalación.
- Procedimiento de puesta en marcha.
- Instrucciones de operación cotidiana.
- Solución de problemas frecuentes.
- Recomendaciones para mantenimiento y actualización.

Apéndice E

Resultados de pruebas

Este anexo debe almacenar tablas, capturas, mediciones y evidencias extendidas derivadas de las pruebas ejecutadas sobre el sistema, especialmente cuando el nivel de detalle excede lo conveniente para el cuerpo principal del documento.

El placeholder actual deja lista una sección para organizar resultados cuantitativos y cualitativos, comparaciones entre escenarios y observaciones complementarias sobre el comportamiento del sistema durante la validación.

Puntos sugeridos para completar

- Tablas de métricas y mediciones crudas.
- Capturas o evidencias visuales de ejecución.
- Comparación entre escenarios de prueba.
- Observaciones adicionales y anomalías detectadas.
- Trazabilidad entre resultados y protocolos aplicados.